

C PROGRAMMING

Handwritten Notes

VIVEKANAND VERNEKAR

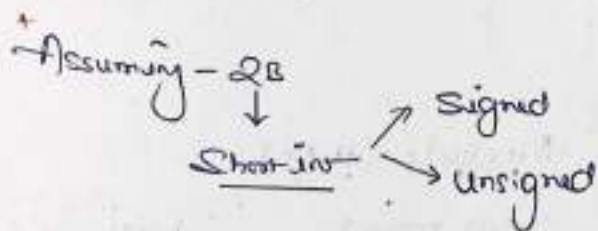
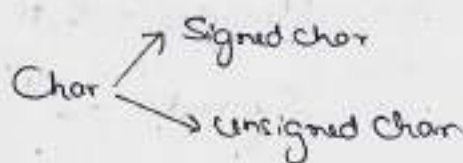
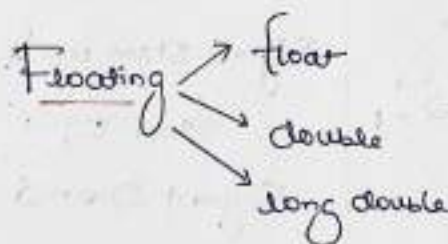
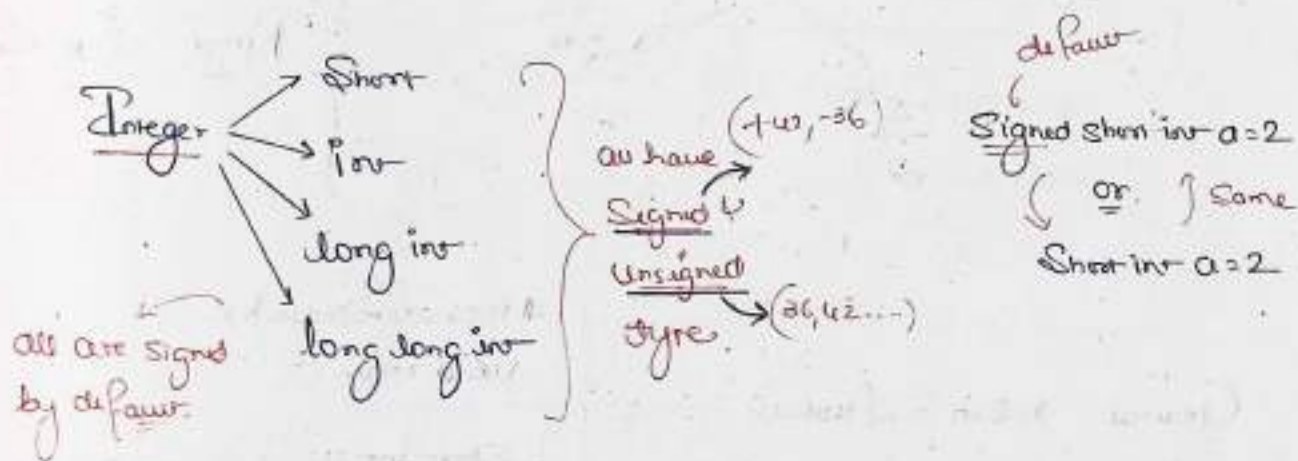
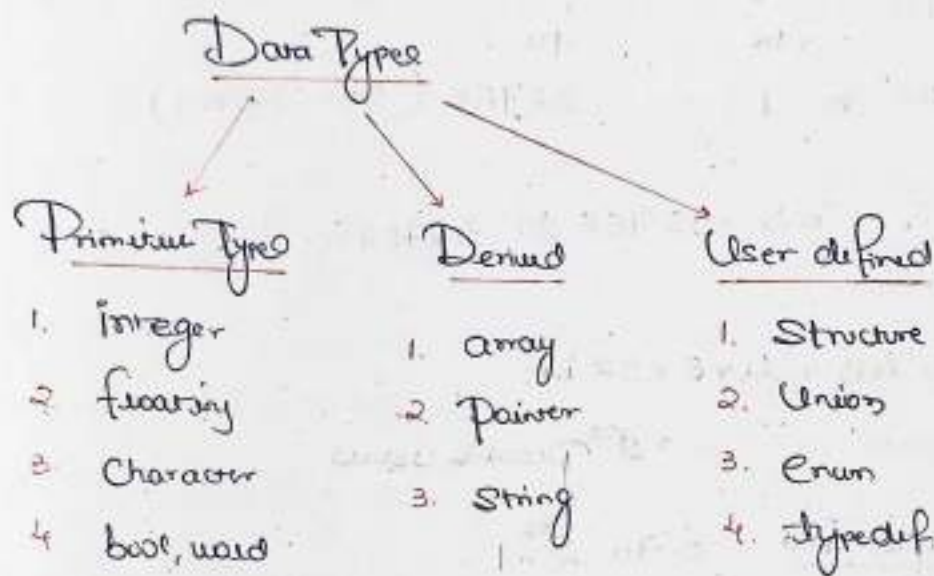
GateWallah Parakram C 2024

(Hinglish Weekday)

Other Notes Uploaded Below

<https://t.me/pwgatenotes>

Data Types and Operators



$$2 \text{ byte} = 2 \times 8 = 16 \text{ bit}$$

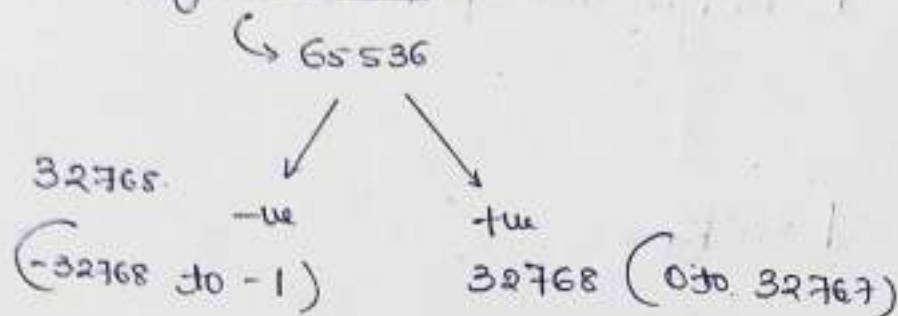
$$= 2^{16}$$
$$= 65,536$$

(each bit stores 0 or 1, (2 possible values))

↓

$$\text{Unsigned short int} = (0 \text{ to } 65,535)$$

Signed Short int

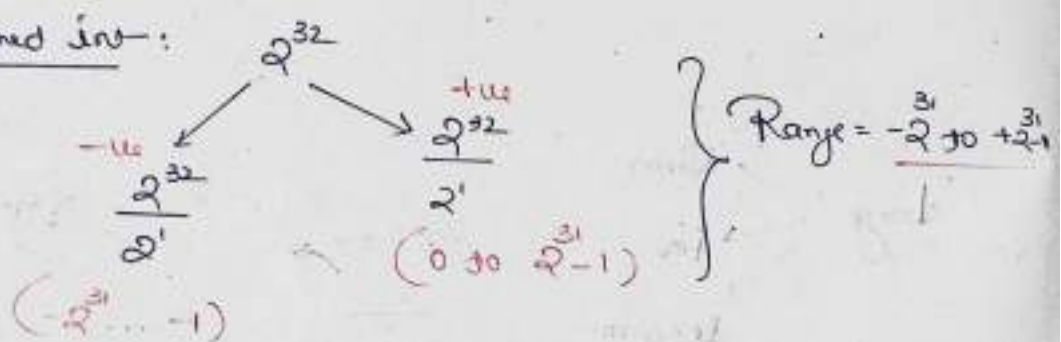


$\Rightarrow -32768$ to 32767 } Range

int $\Rightarrow 4B = 4 \times 8 = 32$ bits
 $= 2^{32}$ possible values

Unsigned int : 0 to $2^{32}-1$

Signed int :



General : n bits = 2^n values

Unsigned = 0 to 2^n-1

Signed = -2^{n-1} to $+2^{n-1}-1$

#include <stdio.h>

void main ()

{ Short int a;

Signed Short int b;

Short c;

Signed Short d;

}

all are
Signed Short
int type
all valid
declarations

#include <stdio.h>

void main ()

{ Signed int a=10; /initialised

int b; }

declared
(has garbage value)

#include <stdio.h>

void main ()

{ Short int a=20;

printf("a"); }

empty inside ""
is considered as text

to print value is a

printf("%d", a)

format
specifier

```
#include <stdio.h>
```

```
void main()
```

```
{ short int a = 20;
```

```
printf("The value is %d", a);
```

```
}
```

O/p:

The value is 20

format specifier of
integer type (short int and int type
not for LL)

```
#include <stdio.h>
```

```
void main()
```

```
{ signed short int a = 20;
```

```
printf("%d is %d", a, a);
```

```
}
```

the values are printed accordingly

O/p $\Rightarrow$ 20 is 20

%d $\rightarrow$ short int, int

%ld $\rightarrow$ long int

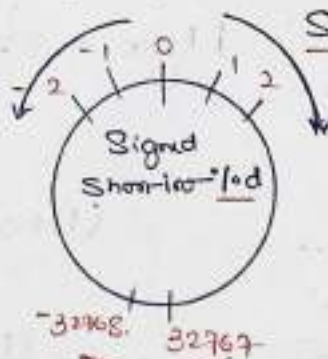
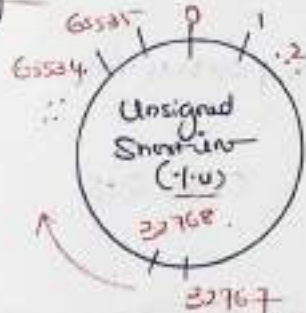
%lld $\rightarrow$ long long int

%c $\rightarrow$ char

%u $\rightarrow$ unsigned short int,
unsigned short
int

Signed int $\rightarrow$ 2 Byte { Cyclic property }

$\rightarrow$ Unsigned 0 to 65535



Signed: -32768 to
+32767

Excess numbers, goes in cycle after limit

Short int i = -2

```
printf("%d", i)
```

O/p = -2

Unsigned short int i = -2;

```
printf("%u", i);
```

O/p = 65534

Short int i = 32769;

```
printf("%d", i);
```

O/p = -32767

because of cyclic
property

Short int i = -32768;

Printf("%i", i);

O/p = 32768

→ goes ahead in cycle after limit.

Unsigned short int a = -3;

Printf("%u", a); / 65535

Printf("%d", a); -3, it's not question

Unsigned, so goes in range of cycle. ∴ 65535

Because printf is

"%d" which prints signed value

Taking input from the keyboard

#include <stdio.h>

void main()

{ int a;

Printf("Enter a number");

Scanf("%d", &a);

Printf("The value of a is %d", a);

}

&a

↓
"address of a"

format to store value at particular address of a variable

Char

1 byte = 8 bits = 2^8
= 256 possible values

Signed Char

256

128

(-128 to -1)

128

(0 to 127)

Range = -128 to 127

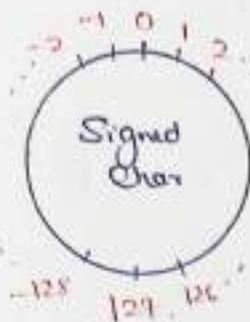
Unsigned Char = 0 to 255

→ Using a Character System, we represent symbols of a lang by two integers.
Constants

A - 65	a - 97	'0' - 48
↓	⋮	⋮
Z - 90	z - 122	

ASCII System.

$\%c \rightarrow$ Character System



Char c = 65;

Printf("%d", c);

Signed int
O/p = 65

Printf("%c", c);

Char
O/p = A
has ASCII value = 65

Signed by default.

Char c = 128;

Printf("%d", c);

Signed int, but 128 cannot be stored in 'char' data type, so goes in cycle due to cyclic property

O/p = -128

Char c = -130;

Printf("%d", c);

Signed integer, -130 not available
so goes in reverse cycle

O/p = 126

decimal value

Char	Stored value
-130	126 $(-130 + 256)$
-132	124 $(-132 + 256)$
-191	65 $(-191 + 256)$

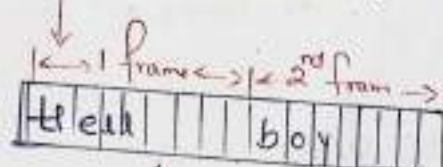
Char c = -191

Printf("%c", c);

O/p = A
because value 65 is stored because of cyclic property

"\n" → Moves the cursor to the beginning of next line.

"\t" → Moves the cursor to next empty available frame.



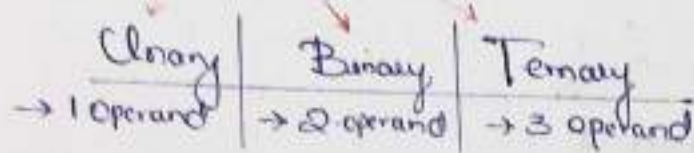
Printf("Hell\tboy");

Each frame is of 8 characters & space in memory

→ A frame is group of 8 characters.

31/10/2023

Operators



1) Binary Operators (2 operands)

→ Assignment Operator:

$a = 30 * 2 + 1$
 evaluate value & then store

Variable = Expression

expression/constant/variable

- Cannot be Expression
- Cannot be Constant
- must be some variable.

→ Arithmetic Operator

$+, -, *, /, \%$
 Math Modulus

* Follows BODMAS Rule & Priority (precedence)

- (i) $\%, *, /$ ↓ Priority high to low
 (ii) $+, -$

eg: $8/2+4$ → both operators same priority

Here Associativity comes into picture. (left → Right)

$a \% b$: returns the remainder when a is divided by b

* Both operand must be of integer type

* Sign of result is same as sign of first operand

→ eg. `int a,b,c;
a=b=c=4;`

`printf("%d", a);` → 4

Result of an Operator $\rightarrow$ Operand

eg: $12/5 \rightarrow 2$

eg: $12.0/5 \rightarrow 2.4$

$12/5.0 \rightarrow 2.4$

$12.0/5.0 \rightarrow 2.4$

eg: `int a;`

`a = 4.0 * 6 % 3 + 2;`

`printf("%d", a);`

$\rightarrow$ Compiler error

$\rightarrow 4.0 * 6 \% 3 + 2$

$= 24.0 \% 3$

Compiler error

Relational Operator

greater

$>, <, >=, <=, ==, !=$

smaller

equal to

not equal to

The result of every relational operator is either 0 or 1.

high

Unary $+, -$

Arith. $[*, /, \%]$ Left to Right
 $[+, -]$

Relational $[<, >, <=, >=]$ Left to Right
 $[==, !=]$ associative.

low

Assignment $=$ Right to Left

Q1) `int i;`

`i = printf("%d", 4/2 + 3*2);`

`printf("%d", i);`

O/p: 8

Q2) `int a;`

`a = printf("%d", printf("Gate
2023"));`

`printf("%d", a);`

O/p: Gate 20239

Logical Operators

- (i) Logical And (&)
 - (ii) Logical Or (||)
 - (iii) Logical Not (!)
- Binary
- Unary

→ `printf("%d", 2 & 7);` ⇒ 1

non zero (true) non zero (true)

All non zero values are treated as "true".

And zero is treated as false.

→ `printf("%d", 2 & -13);` ⇒ 1

→ `printf("%d", 1 & 0);` ⇒ 0

false so result is zero

→ `int a; a = 3 & printf("Hello");
printf("%d", a);` ⇒ Hello 1.

(true)

!(true) = false

→ `printf("%d", 13 & 0.0);` ⇒ 1

→ `int a;`

`a = 0 &`

`printf("%d", a);`

→ If the first operand is 0 for logical And operator then we need not to evaluate 2nd operand.

"Short Circuit Evaluation".

→ `int a;`

`a = 0 & printf("Hello");`

first operand is zero

not evaluated

→ `int a;`

`a = 10 ||   ;`

If first operand

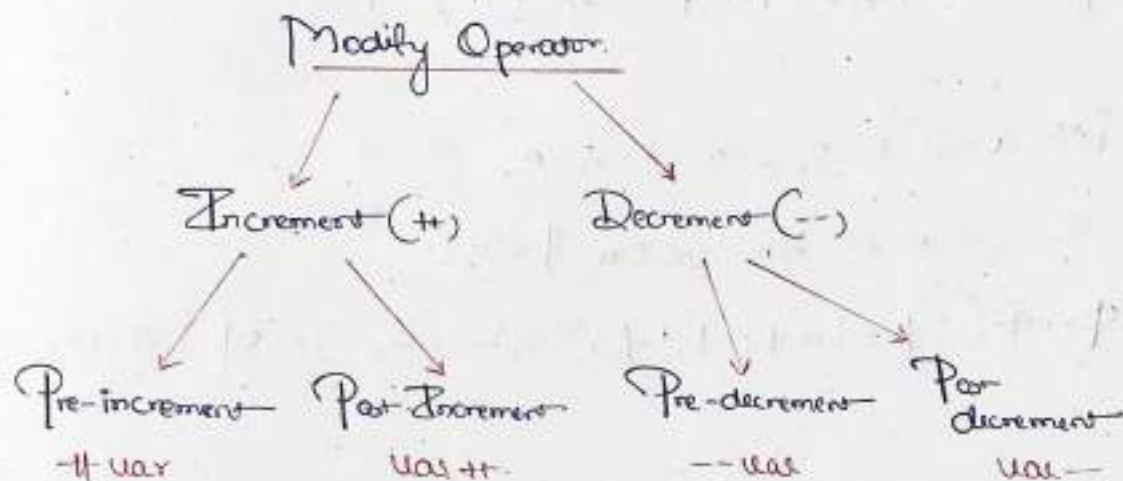
not evaluated

"Short-Circuit Evaluation"

for logical & operators

non-zero, cell need not to

evaluate 2nd operand.



eg: `int a=5, b;`

`b = ++a;`

`printf("%d %d", a, b);`

O/p: 6 6

— first increment the value then
assign/use the value.

`int a=5, b;`

`b = a++;`

`printf("%d %d", a, b);`

O/p: 6 5

— first use the value of var
— then increment the value

Note: In C standard between any two sequence points, a variable can be modified using modify operator only once. More usage of modify operator results in compiler dependent output.

① `int a=0, b=1, c;`

`c = (a & b) + b;` *Show circuit evaluation*

`printf("%d %d %d", a, b, c);` *O/p = 110*

② `int a=0, b=1, c=2, d;`

`d = (a & b) + b;`

`printf("%d %d %d %d", a, b, c, d);` *1031*

③ `int a=2, b=2, c=0, d=2, e;`

`e = ((a & b) + b) + c;`

`printf("%d %d %d %d %d", a, b, c, d, e);` *O/p = 33181*

Decimal to Binary

$(67)_{10} \rightarrow ()_2$

$(1000011)_2$

2	67	Rem
2	33	1
2	16	1
2	8	0
2	4	0
2	2	0
2	1	0
	0	1

Stop here

Binary to Decimal

$$\begin{aligned}
 (1000011)_2 &= 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\
 &= 64 + 2 + 1 \\
 &= 67
 \end{aligned}$$

$$10 \dots \boxed{1} = 2 \times 2 + 1 \rightarrow \text{odd}$$

$$\dots \boxed{0} = 2 \times 2 + 0 \rightarrow \text{even}$$

Bitwise Operators

(i) Bitwise AND (&)

Character by bit

$$\begin{array}{r} a \quad 00000111 \\ b \quad 00001011 \\ \hline \quad 00000011 \end{array}$$

1 & 1 = 1
else all 0

int a=9, b=11, c;

C = a & b;

printf("%d", c);

Op = 3

$$0 \& 0 = 0$$

$$0 \& 1 = 0$$

$$1 \& 0 = 0$$

$$1 \& 1 = 1$$

(ii) Bitwise OR (|)

$$\begin{array}{r} 00000001 \\ 00001001 \\ \hline \quad 11001 \end{array}$$

int a=9, b=17, c;

C = a | b;

printf("%d", c);

Op = 25

$$0 | 0 = 0$$

$$0 | 1 = 1$$

$$1 | 0 = 1$$

$$1 | 1 = 1$$

(iii) Bitwise XOR (^)

int a=5, b=9, c;

C = a ^ b;

printf("%d", c);

Op = 12

$$\begin{array}{r} 00000101 \\ 00001001 \\ \hline 00001100 \end{array}$$

$$\quad 12$$

$$0 \wedge 0 = 0$$

$$0 \wedge 1 = 1$$

$$1 \wedge 0 = 1$$

$$1 \wedge 1 = 0$$

$$a \wedge 0 = a$$

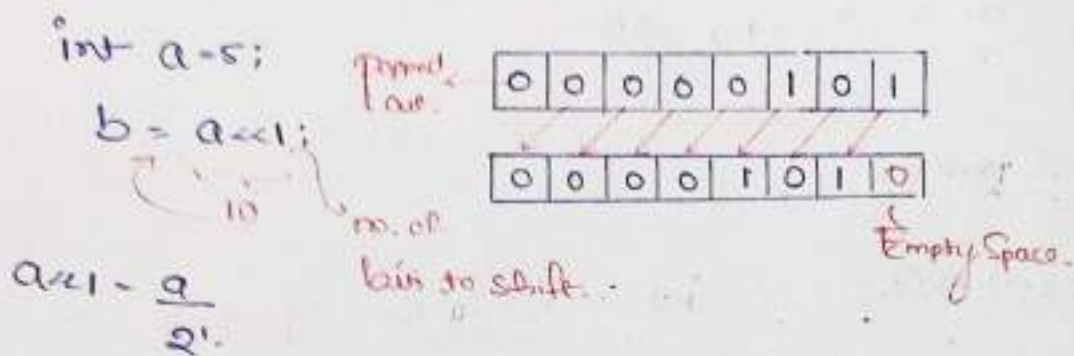
$$a \wedge a = 0$$

$$a \wedge a \wedge a = a$$

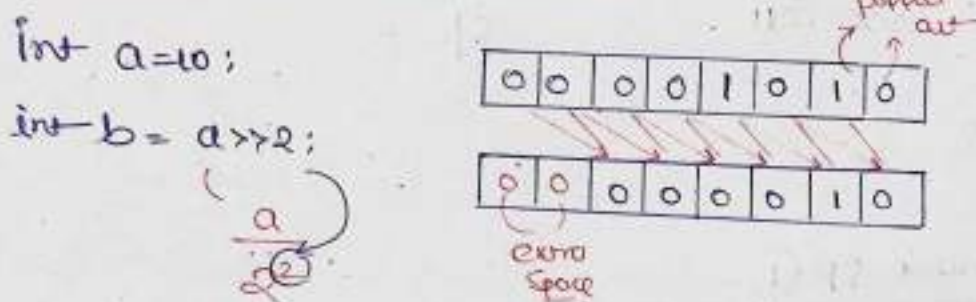
* Even number of times xor the same no. $\Rightarrow 0$

* Odd no. of times $\Rightarrow a$

(iv) Binary Left Shift Operator

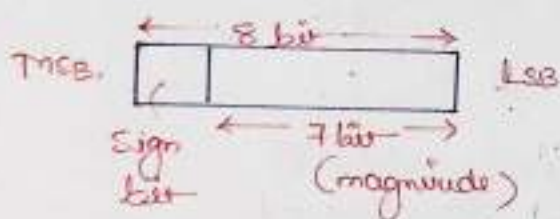


(v) Binary Right Shift Operator

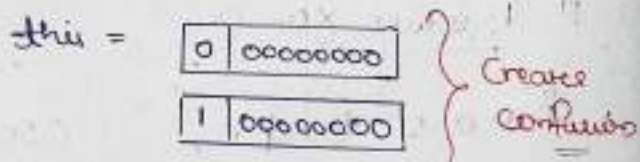


Negative Numbers in Binary

Sign magnitude



Problem with



2's Complement

$$+ve = \text{As it is}$$

$$-ve = 2's \text{ complementation} = 1's \text{ complement} + 1$$

eg:

0101001	
1010110	1's complement
+1	(flip all bits)
<u>1010111</u>	2's complement number

eg: $-23 \Rightarrow 00010111 \rightarrow$ write the positive way
 $\rightarrow \underline{11101001} \rightarrow$ 2's complement. Starting from left keep the first set bit (1) as it is and then flip all the remaining bits

$\rightarrow$ If the number given is 2's complement form

$$\text{msb} \leftarrow 11101001$$

$$2^7 \ 2^6 \ 2^5 \ 2^4 \ 2^3 \ 2^2 \ 2^1 \ 2^0$$

$$\text{Direct method} = -2^4 - 2^2 - 2^1 - 1$$

$$= -23$$

Another approach: Find 2's complement again and then put a negative sign.

(vi) Bitwise Not (~)

int a = 48;

b = ~a;

printf ("%i", b);

$$\boxed{(\sim x) = -(x+1)}$$

$$48 \rightarrow 00110000$$

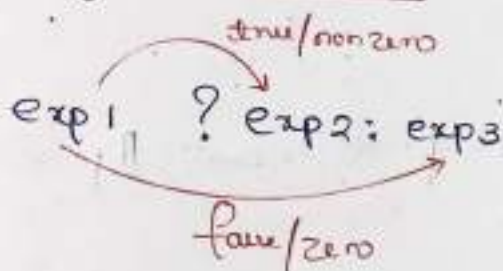
$$\sim 48 \rightarrow 11001111$$

all bits are flipped (2's complement)

$$= -2^5 - 2^4 - 1$$

$$= -49$$

Ternary Operator (?:)



* Expression 1 is evaluated first. If it is true then 'exp2' is evaluated else 'exp3' is evaluated/comes out.

① `int a;`
`a = 12 > 10 ? 12 : 10;`
 true

② `a = 10 ! = 3 > 6 ? 2 == 6 ! 3 ? 10 : 20 : 30;`
 exp1 exp2 exp3
`a = 10`
 true

③ `a = 15 < 3 ? 3 ! = 4 > 10 ? 10 : 20 : 4 < 7 ! = 7 > 10 ? ! 5 ? 30 : ! 2 ! = 2 ?`
 exp1 exp2 exp3
 (false) $\Rightarrow$ exp3
`a = 15 < 3 ? 3 ! = 4 > 10 ? 10 : 20 : 4 < 7 ! = 7 > 10 ? ! 5 ? 30 : ! 2 ! = 2 ?`
 exp1 exp2 exp3
 true
`a = 4 < 7 ! = 7 > 10 ? ! 5 ? 30 : ! 2 ! = 2 ? 4 : 5 : 6`
 exp1 exp2 exp3
 true
`a = ! 5 ? 30 : ! 2 ! = 2 ? 4 : 5 ;` $\rightarrow a = 4$
 exp1 exp2 exp3
 (false)

Shorthand Property:

`x = x + 10` $\rightarrow$ `x += 10`

`x = x * 10` $\rightarrow$ `x *= 10`

`x = x ^ 10` $\rightarrow$ `x ^= 10`

`x = x & 10` $\rightarrow$ `x &= 10`

`x = x << 10` $\rightarrow$ `x <<= 10`

Similarly for all arithmetic, because operator

④ `int a;`
`a = 12 > 3 ? printf("Gee") && printf("Wallaah") || printf("Zee")`
 exp1 exp2
`: printf("Sir");`
 exp3
`printf("%d", a);`
 O/p = GeeWallaah1

⑤ int a;

a = 20 * 110 ? 100 : !2! = 3750 ? 200 : 400;

printf("%d", a);

O/p = 400

Control Flow Statements

#include <stdio.h>

```
void main()
{
    S1;
    S2;
    S3;
    S4;
}
```

Sequential
order.

- 1> Selection Statements : if, if-else, switch
- 2> Iteration Statements : loops : for, while, do-while
(Repetitions)
- 3> Jump Statements : Continue, break, return, exit

Selection Statements

```
S1;
S2;
if (condition/expressions)
{
    S3; S4;
}
S5;
S6;
```

if true
S3, S4 will
execute.
all stmts
in the
flow bracket
will execute.

if (condition/expressions)

only first statement has scope of if.

S1; S2; S3; } independent of if.

if statement.


```

1) void main()
{
    printf("stare");
    if (2 < 5) {
        printf("0");
        printf("1");
    }
    printf("2");
}
O/p: stare-012

```

```

2) printf("1");
   if (5 < 2) {
       printf("2");
       printf("3");
   }
   printf("4");
O/p: 134.

```

```

3) int i = 2;
   printf("1");
   if (i < 2) {
       printf("2");
       printf("3");
   }
   printf("4");
}
O/p: 1234

```

```

4) printf("He");
   if (2) {
       printf("prabhu");
       printf("bachalo");
   }
   printf("Is Rawon Se");
O/p: He prabhu bachalo Is Rawon Se.

```

→ if (-2) { ... } ∴ true

if (printf("GATE")) { ... }

```

if (! printf("Pankaj")) {
    printf("sharma");
}
O/p: Pankaj

```

```

if (condition/expression) {
    ...
} else {
    ...
}

```

Associate an alternate block of code when if condition is false.

```

if (12) 0 (false)
    printf("muo");
else if (2+3=5) 0 (false)
    printf("daron");
else
    printf("ye Rawan hai");

```

Given 3 distinct integers
 Get the maximum number

```

int a, b, c, max;

max = (a > b & a > c) ? a :
      b > c ? b : c;

printf("%d", max);

```

Iterative Statements

(i) For loop :

```

for (initialisation; condition; Inc/dec)
{
    code;
}

```

Flowchart for For loop:

1. Initialisation
2. Condition
3. Code
4. Increment/Decrement

if true → go to 3
 if false → again goes to 2 expression & condition is checked if false, break out of loop

```

for (expression 1; expression 2; expression 3)
{
    ... code;
}

```

if no semicolon { }, scope will be available till first statement

eg: `for (int i=1; i<=5; ++i)`
`{ printf("%d", i); }`

O/p: 1 2 3 4 5

```

int i=1;
for (i=10; i<=5; 10)
{
    printf("%d", i);
    i=i+2;
}

```

increment

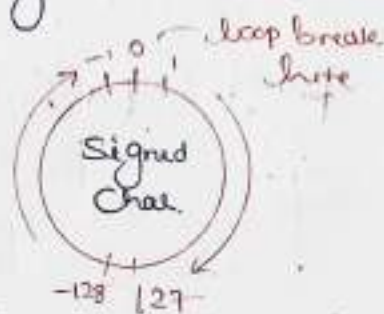
O/p = 1 3 5

① Char ch = 1;

```
for (10; ch; ch++)  
{ printf("Hello"); }
```

255 times

How many times printf will execute?



② Char ch = 1;

```
for (10; ch; ch+=2)  
{ printf("Hello"); }
```

Infinite loop as it doesn't land on 0.

⇒ 255 times

③ int i = 1;

```
for (printf("1"); i <= 4; printf("4"))  
{ printf("2");  
  i = i + 1; }
```

O/P = 124242424

④ for (1; 2; 3)
{ printf("Hello"); }

∞ loop

⑤ for (1; 0; 1)
{ printf("Hello"); }

0 times

int i = 1;
⑥ for (; i <= 5; i++)
{ printf("Hello"); }

Prints 5 times
"Hello"

* All three expressions are optional, but semi colon mandatory.

⑦ for (; ; ;) (true)
{ printf("Hello"); }

∞ loop

⑧ $\text{int } i = 1;$

$\text{for } (i \text{++} < 6;)$ ^{post increment}

$\{ \text{printf}(\text{"\%d", } i); \}$

O/p = 23456

Loop Analysis

1) $\text{for } (\text{int } i = 0; i < n; ++i)$

$\{ \text{printf}(\text{"\%d", } i); \}$ $\rightarrow$ loop runs $\underline{n}$ times.

2) $\text{for } (\text{int } i = 1; i \leq n; i = i + 2)$

$\{ \dots \}$ $\rightarrow$ loop runs $\left\lceil \frac{n}{2} \right\rceil$ times.

3) $\text{for } (\text{int } i = 1; i \leq n; i = i * 2)$

$\{ \dots \}$ $\rightarrow \lfloor \log_2 N \rfloor + 1$ times

4) $\text{for } (\text{int } i = 1; i \leq n; i = i * 3)$

$\{ \dots \}$ $\rightarrow \lfloor \log_3 N \rfloor + 1$ times

5) $\text{for } (\text{int } i = 1; i \leq n; ++i)$

$\{ \text{for } (\text{int } j = 1; j \leq n; ++j)$

$\{ \text{printf}(\text{"\%d", } i); \}$

$\}$

$\Rightarrow n \times n = n^2$
times

n times

n times

6) for (int i=1; i ≤ n; ++i)

{ for (int j=1; j ≤ n; j = j*2)

{ printf("pankaj"); }

$$= n (\lfloor \log_2 n \rfloor + 1)$$

7) for (int i=1; i ≤ n; i = i+2)

{ for (int j=1; j ≤ n; j = j*2)

{ printf("pankaj"); }

$$\Rightarrow \left\lceil \frac{n}{2} \right\rceil (1 + \lfloor \log_2 n \rfloor)$$

8) for (int i=1; i ≤ n; ++i)

{ for (int j=1; j ≤ i; ++j)

{ printf("pankaj"); }

i=1 | i=2 | i=n
j=1 | j=1,2 | j=1,2,...,n

$$\rightarrow n \frac{(n+1)}{2}$$

9) for (int i=1; i ≤ n; ++i)

{ for (int j=i; j ≤ 3*i; ++j)

{ ... }

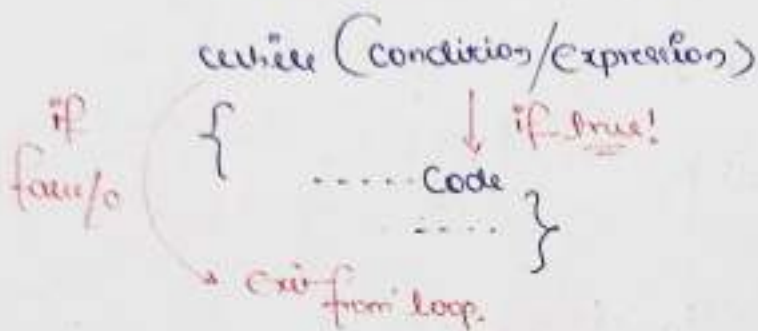
i=1 | i=2
j=1 to 3 | j=2 to 6 ...
3 times | 5 times ...

i=n
j=n to 3n = 3n-n+1

$$(2n-1) \text{ times}$$

$$\begin{aligned} [3+5+\dots+(2n-1)] &= \text{AP} \\ &= \frac{S_n}{2} = \frac{n}{2} (2a + (n-1)d) = \frac{n}{2} (3 + (2n-1)) \\ &= \frac{n}{2} (2n+2) \\ &= n(n+1) \end{aligned}$$

(ii) While loop



loops → for :- no of iterations are known in advance.
while :- when no of iterations are not known.

→ WAP to calculate the no. of digits in a number.

```
int C = 0;  
while (n != 0)  
{ C++; → count digit  
  n = n/10; } → delete last digit  
return C;
```

(ii) do-while loop :

```
int i = 1;  
do { printf("Pankaj");  
    }  
while (i > 5);
```

O/P = Pankaj.

* The code will execute the code in loop atleast one time irrespective of whether the condition is true or false.

Jump Statements

(i) Continue:

```
for ( ; ; )  
{  
    S1;  
    S2;
```

```
    Continue;  
    S3;  
    S4;  
}
```

 } Skip the remaining part of current iteration & Continue with next iteration.

```
for (int i=1; i<=10; ++i)
```

```
{  
    if (i%3 == 0)  $\Rightarrow$  in this case  
        Continue; } below part is skipped  
    printf("i.d", i);  
}
```

O/p = 1 2 4 5 7 8 10

(ii) Break:

```
for (int i=1; i<=10; ++i)
```

```
{  
    if (i%3 == 0)  $\rightarrow$  when the condition is  
        break; } true, break is called when  
    printf("Hello"); } breaks the loop when  
                        i=3.  
                        HelloHello.
```

```
for (int i=1; i<=5; ++i)
```

```
{  
    for (int j=1; j<=5; ++j)
```

```
{  
    if ((i+j)%4 == 0)
```

```
        break; } - break the inner loop
```

```
    printf("i.d j.d", i, j);  
}
```

```
}
```

O/p = 1 1 2 2 1 4 1 4 2 4 3 5 1 5 2

$\text{for } (\text{int } i=1; i \leq n; i=i*3)$ $i=1$ $i=3^1$... $i=3^k$
 $\{$ $j=1 \text{ to } n$ $j=3 \text{ to } n$ $j=3^k \text{ to } n$
 $\text{for } (\text{int } j=i; j \leq n; j++)$ $(n-1)+1$ $(n-3+1)$ $(n-3^k+1)$
 $\{ \text{printf}("uuu"); \}$
 $\}$

$$= (k+1)(n+1) - \frac{(3^{k+1}-1)}{3-1}$$

$$= (k+1)(n+1) - \frac{(3^{k+1}-1)}{2}$$

$$= (\lfloor \log_3 n \rfloor + 1)(n+1) - \frac{(3^{\lfloor \log_3 n \rfloor + 1} - 1)}{2}$$

$$3^k \leq \log n$$

$$\log 3^k \leq \log n$$

$$k \log 3 \leq \log n$$

$$k \leq \frac{\log n}{\log 3}$$

$$k = \lfloor \log_3 n \rfloor$$

$\text{for } (\text{int } i=1; i \leq n; ++i)$

$\{ \text{for } (\text{int } j=i; j \leq i+i; j++)$

$\{ \text{for } (k=1; k \leq n; k++)$

$\{ \text{printf}("pouka"); \}$
 $\}$

$$\rightarrow \sum_{i=1}^n \sum_{j=1}^{i+i} \left(\sum_{k=1}^n 1 \right)$$

Switch Statement

* Switch is a keyword

* Used to provide Selection Statement with multiple choices

* Multiple choices are provided using keyword : case

Switch(n)

{ Case (1): Code to be executed when value of n is 1
break;

Case 2: Code to be executed when value of n is 2
break;

default: Code to be executed when none of above cases match.
break;

}

Switch (Expression) Expression that evaluated to be its value.

(5)

↑ allowed because it has a value.

{ Case Constant1: Code1
break; } Switch
break; → breaks the loop if matched.

Case Constant2: Code2:
break;

* Continue is only for loops not for switch.

Case Constant3: block of code
break;

* if break is not there then cases below will execute (default).

Case labels cannot be a variable. It can only contain constants.

default: Code;
break;

→ takes out of the scope "break" is optional.
fall through will happen.

}

* duplicate case labels are not allowed.

i = 2;

eg: Switch (i+2)

{ case 2: printf("2");
break;

printf("hello"); → ignored.

case 4: printf("4");

}

Note:

- * Break is optional
- * Order of case labels does not matter.
- * Position of default does not matter.
- * Default is optional.
- * Switch(i); ✓
- * Switch(i) { } ✓
- * Switch(); ✗ Compiler error, Expression is mandatory
- * Case labels must be constant or any expression containing all constants.
- * Duplicate case labels not allowed

eg: low, high
code:

```
int i = 3;
Switch(i)
{
    Case 1...10 : printf("Hello");
                 ↓
                space
                 break;
    Case 11...20 : printf("World");
                 ↓
                3 dots
                 break;
}
```

* Not for all Compilers

Any statement here is
ignored.

O/p: Hello

Functions and Storage Classes

* Compilation happens top to bottom.

```
#include <stdio.h>
```

```
int multiply (int, int);
```

→ forward declaration, to avoid C.E. as we are calling the function and defined later or below

```
void main()
```

```
{ int a=10, b=20, answer; // int type
```

```
    answer = multiply(a,b); // call/use of function
```

```
    printf ("%d", answer);
```

if return type is int, data not match. Error!

return type of function

```
int multiply (int x, int y)
```

→ function header

```
{ int temp;
  temp = x*y;
  return temp; }
```

function definition

if not written anything, by default

Compiler will set return type as 'int', even in

also in forward declaration!

(Mismatch)

eg: void main()

```
{ int a=10;
```

```
  double b;
```

```
  b = fun(a);
```

→ Compiler cause info = return type is integer

```
  printf ("%f", a);
```

```
  return 0;
```

```
}
```

Return type of fun is int

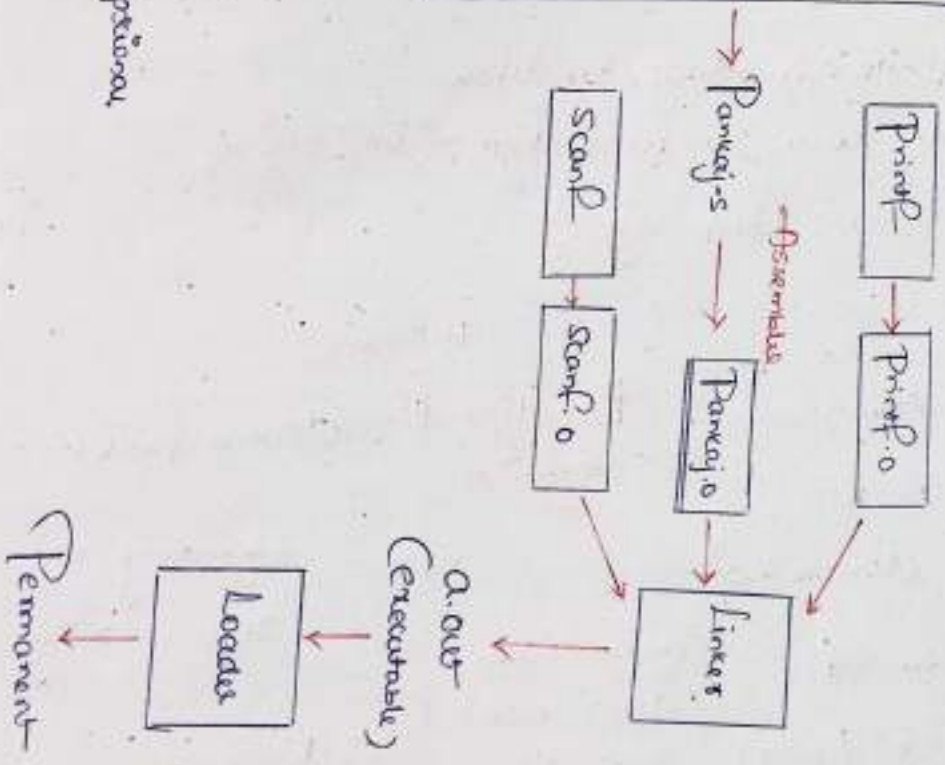
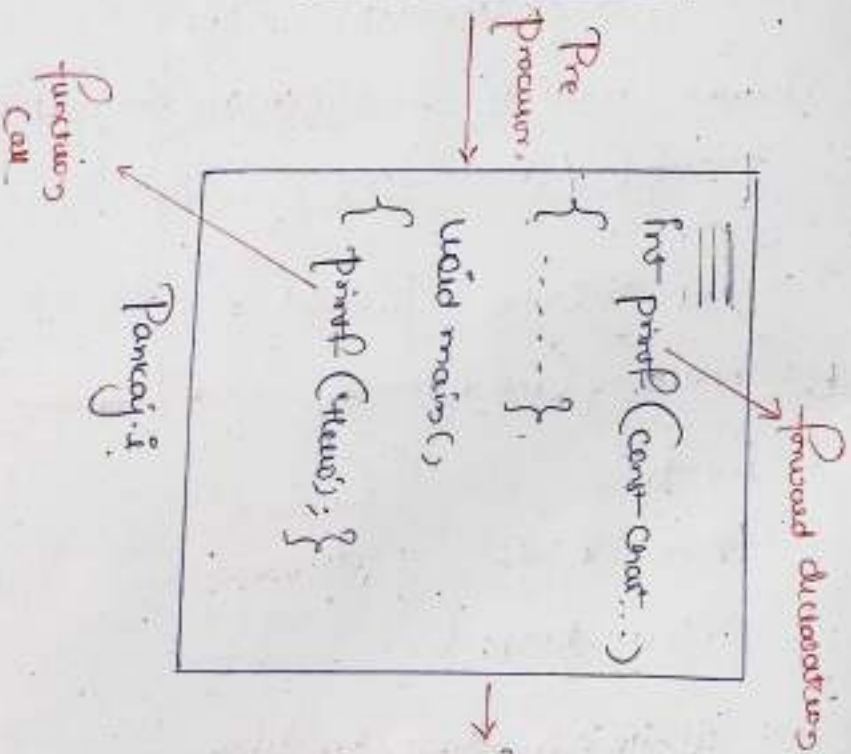
```
double fun (int x)
```

```
{ double y = 10.2;
  return x*y; }
```

∴ Error

```
#include <stdio.h>
void main()
{
    printf("Hello");
}
```

Pankaj.c



Collecting/Storing the returned value from a function is optional.


```
#include <stdio.h>
```

```
void main()
```

```
{ int a=10, b=20;
```

```
  mul(a,b); }
```

→ function called but value returned
by the function not stored
→ no error

```
int mul(int x, int y)
```

```
{ int temp;
```

```
  temp = x+y;
```

```
  return temp; }
```

How a function works

```
#include <stdio.h>
```

```
int add(int, int);
```

```
void main()
```

```
{ int a=10, b=20, ans;
```

```
  ans = add(a, b);
```

```
  printf("%d", ans);
```

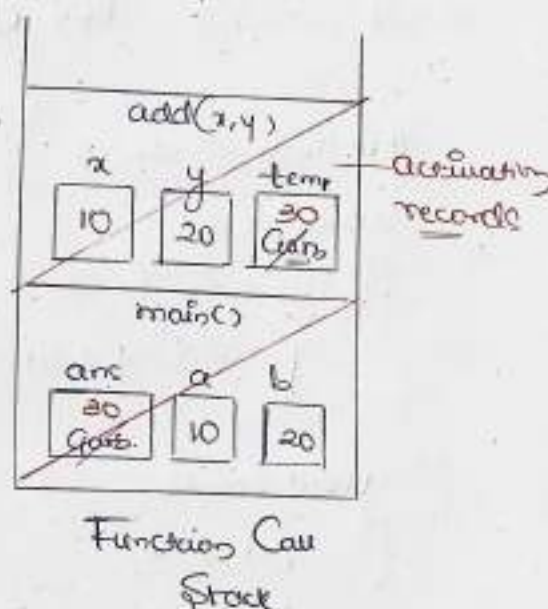
```
}
```

O/p = 30

```
int add(int x, int y)
```

```
{ int temp = x+y;
```

```
  return temp; }
```



```
#include <stdio.h>
```

```
void swap(int, int);
```

```
void main()
```

```
{ int a=10, b=20; // O/p = 10 20
```

```
  printf("%d %d", a, b);
```

```
  swap(a, b); // actual arguments
```

```
  printf("%d %d", a, b);
```

```
}
```

formal parameters

```
void swap(int x, int y)
```

copy of the arguments passed will be here. So variables in main will not be changed.

```
{ int temp;
```

called by values

```
  temp = x;
```

$x = y$

```
  y = temp; }
```

Storage Class

Scope: part of the program in which a variable is visible.

It is the section of the code in which the variable can be accessed directly.

Lifetime: duration $\rightarrow$ alive.

Storage area: where a variable is stored.

Default Value: The value that is present in a variable when we don't initialise the variable.

```
#include <stdio.h>
```

```
void f()
```

```
{ int a=10, b=20;  $\rightarrow$  Start of scope  
  printf("%d, %d", a, b);  $\rightarrow$  local variables  
}
```

```
void main()
```

```
{ int x=10;
```

```
  f();  $\rightarrow$  function calling.
```

```
  printf("%d", x);  $\rightarrow$  Error, cannot access local variable of another scope.
```

Error, cannot access local variable of another scope.

Auto Variables

- * By default, variables declared inside a function are auto variables.
- * Scope and block: block in which they are declared.
- * Default Value: Garbage.
- * Storage area: Stack.

```
void main()
```

```
{  
  int a = 0;  
  ++a;  
  printf("%d", a); // O/P = 1  
}
```

```
    {  
      int a = 10;  
      ++a;  
      printf("%d", a);  
    } // New Scope (Sub Scope)
```

```
    ++a;  
    printf("%d", a);  
  }  
  // O/P = 2  
  // Overall Output: 1112
```

- * Main Variable Scope Variable can be accessed to sub-scope but not vice versa
- * Sub-scope variable are not accessible to main scope
- * Auto variables are created automatically when we enter the block in which they are declared and destroyed automatically when we exit the block.

eg: #include <stdio.h>

```
void f1()
```

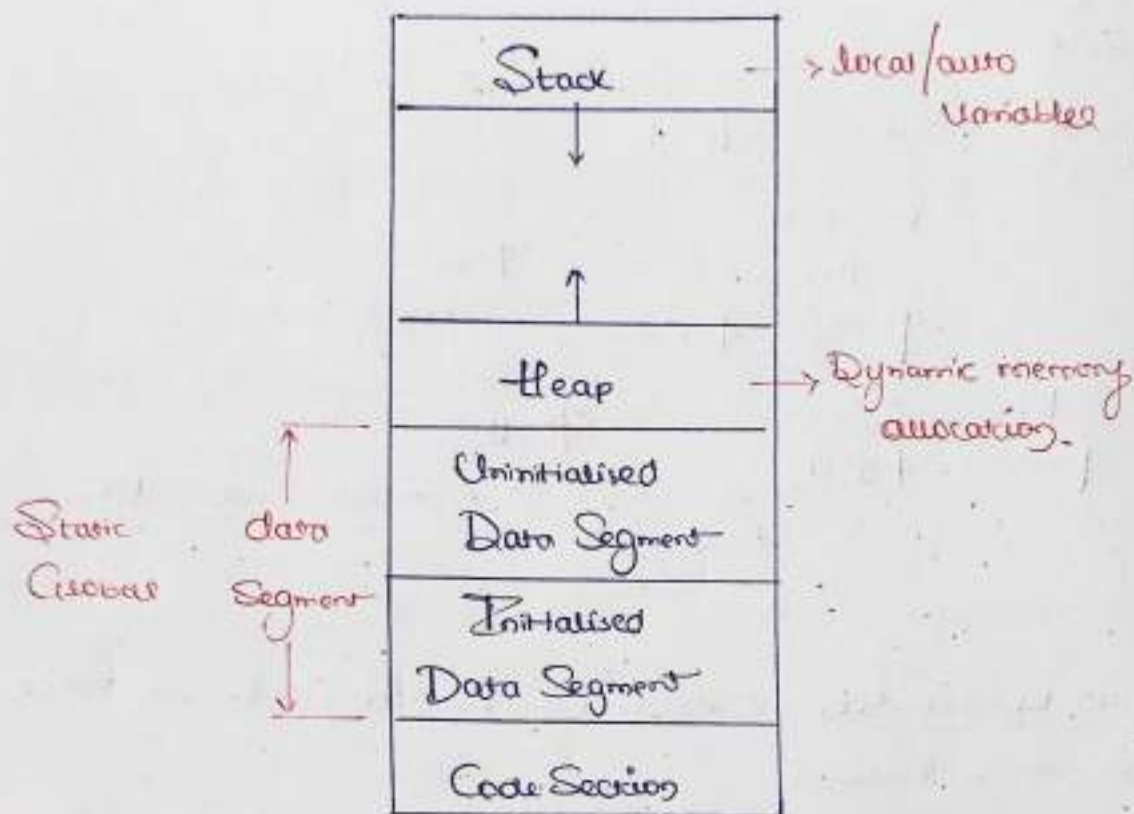
```
{  
  int a = 0;  
  ++a;  
  printf("%d", a);  
}
```

```
void main()
```

```
{  
  f1;  
  f1;  
  f1;  
}
```

Output = 111

Memory Diagram

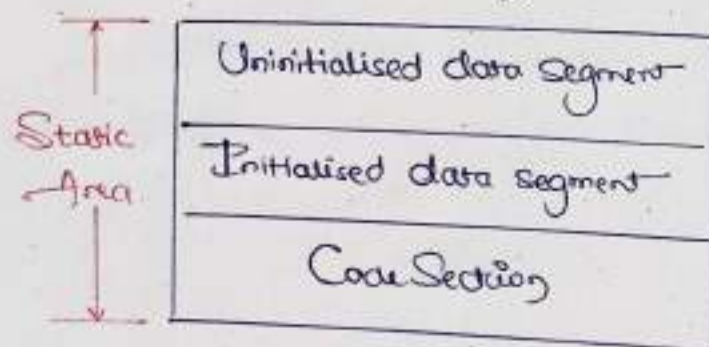


Register

- * Same as auto variable
- * Storage area: CPU register/stack

Static Variables

- * Scope: block in which they are created
- * lifetime: lifetime of program.
- * default value = 0
- * Storage area: Static area (data segment)



```
#include <stdio.h>
```

```
void fun()
```

```
{ static int i=0; } // this line is executed only once
```

updates the last updated value.

```
printf("%d", i);
```

```
void main()
```

```
{ fun(); // O/p = 123.  
  fun();  
  fun(); }
```

* No redeclarations

* Values present b/w different functions call

* They are created only once in the program.

Global Variable

* By default, a variable defined outside all the functions is called as global variable.

* Global variables can be accessed inside every function if the variable is declared above the function definition.

```
void fun()  
{ ++x;  
  printf("%d", x); }
```

int x=0; ← global variable

```
void main()  
{ ++x;  
  fun();  
  printf("%d", x); }
```

```
void fun()  
{ extern int x; → local forward declaration for  
  ++x;  
  printf("%d", x); }
```

int x=0; * extern is only applicable on global variable

```
void main()  
{ ++x;  
  fun();  
  printf("%d", x); }
```

extern int x; → global forward declaration

```
void fun() { ... }
```

```
void main() { ... }
```

int x=0; → global variable definition

Forward
Linkage

```
Static int i=10;  
void f() { }  
void g() { }  
void main()  
{ }
```

```
int i=10;  
void f() { }  
void g() { }  
void main()  
{ }
```

Forward
Linkage

Forward declaration

```
#include <stdio.h>  
void main()  
{ printf("Hello"); }
```

Pankaj.c

Pre
Processing

```
extern int printf(...);  
  
void main()  
{ printf("Hello");  
}
```

PF-definition

Linear

PF-fd
PF-cou

Recursion:

The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is a recursive function.

```
{ if (n is small)  
{ - Can be answered directly.  
  - no recursion needed.  
  - easy problem. }  
else { - Cannot be answered directly.  
      - not easy  
      - recursion needed. }  
}
```


eg: Print "Pankaj" n times using recursion.

```
void Print(int n)
{
    if (n == 1) printf("Pankaj");
    else { printf("Pankaj");
          Print(n-1); }
}
```

↓
Recursively call for smaller values.

eg: Sum of digits using recursion.

```
int sum_digits(int n)
{
    if (n > 0 && n <= 9)
        return n;
    else { return (n/10) + sum_digits(n/10); }
}
```

eg: 125

125 / 10 = 12
= 5 + sum_digits(12)

12 / 10 = 2 + sum_d(12/10)

12 / 10 = 1 + sum_d(12/10)

12 / 10 = 1 + sum_d(12/10)

12 / 10 = 1 + sum_d(12/10)

eg: a^b $a > 0$ $b \geq 0$

```
int Power(int a, int b)
{
    if (b == 0) → base condition
        return 1;
    else { return a * Power(a, b-1); }
}
```

$a=3$ $b=3$ 27

$3 \times \text{Power}(3, 2) \leftarrow a$

↓

$3 \times \text{Power}(3, 1) \leftarrow 3$

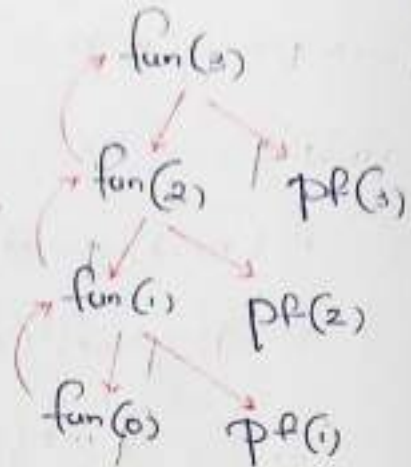
↓

$3 \times \text{Power}(3, 0) \leftarrow 1$

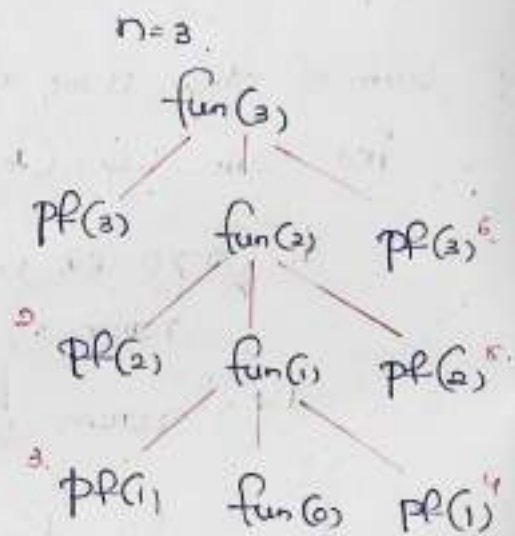
↓

return 1

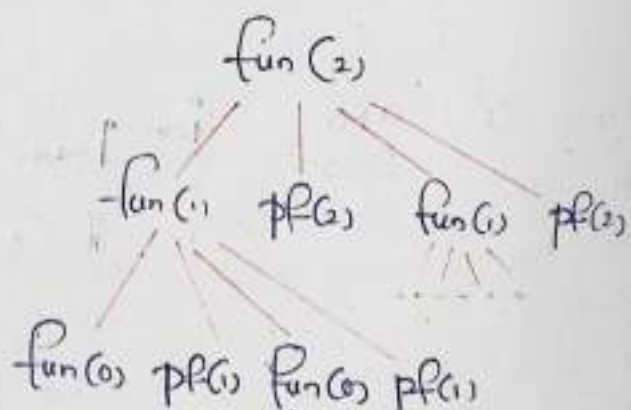
eg: void fun(int n)
 { if (n <= 0) return;
 else { fun(n-1);
 printf("%d", n); }
 }
 O/P = 123.



eg: void fun(int n)
 { if (n <= 0)
 return;
 else { printf("%d", n);
 fun(n-1);
 printf("%d", n); }
 }
 O/P = 321123.



eg: void fun(int n)
 { if (n <= 0)
 return;
 else { fun(n-1);
 printf("%d", n);
 fun(n-1);
 printf("%d", n); }
 }
 O/P = 112112



① void f(int n)

```
{ if (n <= 0)
    return;
```

```
printf("%d", n);
```

```
f(n-1); }
```

What is o/p of f(5)?

→ 54321

② void f(int n)

```
{ if (n <= 0)
    return;
```

```
f(n-1);
```

```
printf("%d", n); }
```

What is the output of f(5)?

→ 12345

③ void f(int n)

```
{ if (n <= 0) return;
```

```
f(n-1);
```

```
printf("%d", n);
```

```
f(n-1); }
```

What is o/p of f(4)?

→ 121312141213121

④ int f(int n)

```
{ if (n <= 1)
```

```
    return 0;
```

```
    return f(n/2) + f(n/2 + 1); }
```

What is the output of f(5)?

→ 7

⑤ int f(int n)

```
{ if (n <= 1)
```

```
    return n;
```

```
    return f(n/2) + n/2; }
```

What is o/p of f(2)?

→ 11

⑥ int f(int n)

```
{ if (n <= 1) return n;
```

```
    if (n % 2)
```

```
        return f(n/2) + n;
```

```
    return f(n/3) + n;
```

```
}
```

What is the output of f(22)?

→

⑦ // Assume $n \geq 0$

```
void fun(int n)
{
    if (n == 0) return;
    fun(n/2);
    printf("%d", n/2);
}
```

What is O/p of $f(11)$?
→ 1011

```
⑨ void main()
{
    static int var = 5;
    printf("%d", var--);
    if (var)
        main();
}
```

```
⑩ void main()
{
    static int i = 5;
    if (--i)
    {
        main();
        printf("%d", i);
    }
}
```

O/p = 0000

```
⑧ void foo(int n, int sum)
{
    int k = 0, j = 0;
    if (n == 0) return;
    k = n * 10;
    j = n / 10;
    sum = sum + k;
    foo(j, sum);
    printf("%d", k);
}
```

```
void main()
{
    int a = 2018, sum = 0;
    foo(a, sum);
    printf("%d", sum);
}
```

Output is 20480

```
⑪ int fun(int a, int b)
{
    if (b == 0) return 0;
    if (b % 2 == 0)
        return fun(a + a, b/2);
    return fun(a + a, b/2) + a;
}
```

```
int main()
{
    printf("%d", fun(4, 3));
    return 0;
}
```

Output : 12

⑫ `int f (int n)`
`{ static int r=0;`
`if (n ≤ 0) return 1;`
`if (n < 3)`
`{ r = n;`
`return f(n-2)+2; }`
`return f(n-1)+r; }`

What is the value of $f(5)$?
 → 9

⑬ `int fun (int x)`
`{ if (x > 3)`
`return fun(x-4) + fun(x-1) + 1;`
`return 1; }`

What is the value returned
 by $fun(12)$?
 → 51

⑭ `int fun (int x, int y)`
`{ if (x == 0)`
`return y;`
`return fun(x-1, x*y); }`

What is the output of
 $fun(4, 3)$?
 → 13.

⑮ `unsigned int foo (unsigned int n,`
`unsigned int r)`
`{ if (n > 0)`
`return (n/r) + foo(n/r, r);`
`else`
`return 0; }`

Output of $foo(513, 2)$?
 → 2.

⑯ `int fun (int n)`
`{ if (n == 4) return n;`
`else return 2 * fun(n+1); }`

`int main()`
`{ printf ("%d", fun(2));`
`return 0; }`
 → 16

⑰ `int fun (int x, int y)`
`{ if (y == 0) return 0;`
`return (x + fun(x, y-1)); }`

What does the following function
 do?
 → x^y .

⑮ `int fun (int x, int y)`
`{ if (y == 0) return 0;`
`return (x + fun (x, y-1));`
`}`

`int fun2 (int a, int b)`
`{ if (b == 0) return 1;`
`return fun (a, fun2(a, b-1));`
`}`

What does `fun2()` do?

→ `pow(x, y);`

⑰ `#include <stdio.h>`
`void print (int n)`
`{ if (n > 4000) return;`
`printf("%d", n);`
`printf(" 2*n");`
`printf("%d", n); }`

`int main()`
`{ print(1000);`
`getchar();`
`return 0; }`

What is the output?

→ 1000 2000 4000 4000 2000 1000

⑳ What does the following function do?

`int fun (unsigned int n)`
`{ if (n == 0 || n == 1) return n;`
`if (n % 3 != 0) return 0;`
`return fun(n/3); }`

→ It returns 0/1 when `n` is a power of 3, otherwise returns 0.

㉑ Predict the output of the following program.

`#include <stdio.h>`
`int f (int n)`
`{ if (n < 1) return 1;`
`if (n % 2 == 0) return f(n/2);`
`return f(n/2) + f(n/2+1); }`

`int main()`
`{ printf("%d", f(5));`
`return 0; }`

→ 5

Q2) `int f(int n)`
`{ static int i=1;`
`if (n < 5)`
`return n;`
`n = n + i;`
`i++;`
`return f(n); }`

Value returned by `f(1)` is?
 $\rightarrow 7$

Q3) `void Count(int n)`
`{ static int d=1;`
`printf("%d", n);`
`printf("%d", d);`
`d++;`
`if (n > 1) Count(n-1);`
`printf("%d", d); }`

`int main()`
`{ Count(3); }`

$\rightarrow 312213444$

Q4) Consider the following recursive C function. If `get(6)` function is being called in `main()`, then how many times will the `get()` function be invoked before returning to the `main()`.

`void get(int n)`
`{ if (n < 1) return;`
`get(n-1);`
`get(n-3);`
`printf("%d", n); }`

$\rightarrow \underline{25}$ (Ans)

Q5) What will be the output of the following C program?

`#include <stdio.h>`
`int main()`
`{ main(); return 0; }`

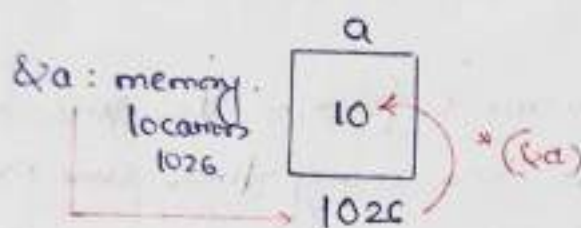
$\rightarrow \underline{\text{Runtime Error}}$

Array and Pointers

Address $\rightarrow$ Absolute address
 $\rightarrow$ Relative address

$\rightarrow$ * : value at operator
 $\rightarrow$ & : address of operator

int a = 10;



*(&a) = value at (memory location 1026) = 10

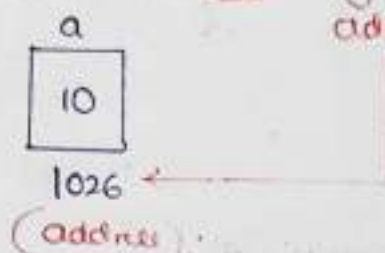
*(&a) = a

How to get address of any variable?

int a = 10;

$\rightarrow$ & : address of operator

&a $\rightarrow$ gives the address of a

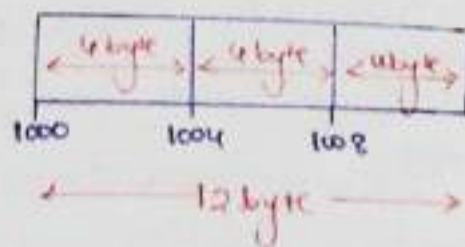


Array: Array can be defined as a group of multiple entities of similar type into a larger group. These entities can be int, float, double, char or can be user defined data types also.

eg. int a, b, c, d, e $\Rightarrow$ int arr[5];

if int = 4 byte

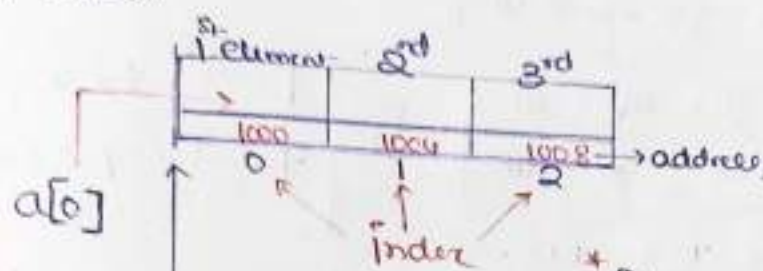
int arr[2]



* All elements are stored sequentially and represented by same name 'arr'.

* In C, index always starts from 0.

`int a[5];`



* Array name represent address of its first element.

* Array name $\rightarrow$ Constant address

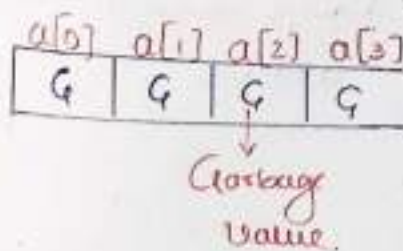
`++ array_name`
`-- array_name`
`array_name ++`
`array_name --`

all are invalid because array name is a constant address.

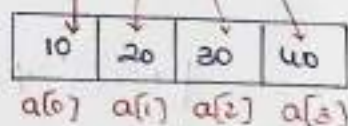
Array name $\nrightarrow$ { } $\times$

Invalid because l-value cannot be a constant.

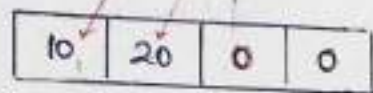
```
void main()
{
    int a[4];
    printf("%d", a[2]);
}
```



`int a[4] = {10, 20, 30, 40};`



`int a[4] = {10, 20};`



`int a = {10};` $\checkmark$ valid

`int a[10];` $\checkmark$

`int a[2*3+10];` $\checkmark$

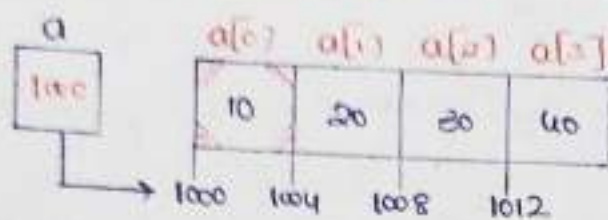
`int a[2*sizeof(int)];` $\checkmark$

`int a[ ];` $\times$ Invalid

`int a[ ] = {10, 20, 30};` $\checkmark$ valid

① Array name represents address of its first element.

$\text{int } a[u] = \{10, 20, 30, 40\}$



`printf("%d", a);`
`printf("%d", &a[0]);`

② Array name does not represent an address with & operator.

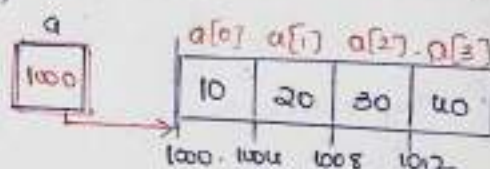
(i) `&` : address of Operator

(ii) `sizeof` : Size of Operator

$\text{int } a[u] = \{10, 20, 30, 40\}$

Address of Element of `a` Byte.

`printf("%d", a);`
`printf("%d", &a[0]);`
`printf("%d", &a);`



Address of whole

Array, 16 byte address.

`printf("%d", &a[0]);`
`printf("%d", &a);`
 Numerical values are same but logically we are talking about different elements' address.

③

Numerical value of `a` is 100

`a+1`

`a` is simple var value

`a` is an address / pointer variable

`int a = 100;`

`printf("%d", a);`

= 101

Address Arithmetic

Address + value = address ✓
 Address + address = invalid ✗

④ If the declaration (Initialization) of an array has n -dimension.

`int a[u];` $\longrightarrow$ 1 dimension

`int a[2][3];` $\longrightarrow$ 2 dimension

`int a[2][3][u];` $\longrightarrow$ 3 dimension

* Anywhere other than declaration if we are provide exactly n -dimension then we are working on element.

* If we are providing less than n -dimension then we are working on address.

eg: `int a[2][3];` $\rightarrow$ dimension

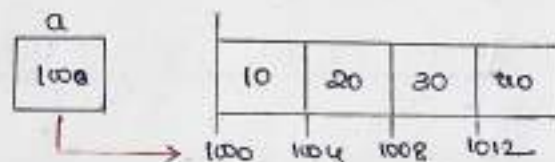
`a;` $\rightarrow$ 0 dimension $\rightarrow$ address

`a[0];` $\rightarrow$ 1 dimension $\rightarrow$ address

`a[0][0];` $\rightarrow$ 2 dimension $\rightarrow$ element

eg: `int a[u] = {10, 20, 30, 40};`

`printf("%u", a+1);`



`a` $\rightarrow$ address



`arrayname = &a[0]`

$\downarrow$
Size = 4 byte.

$$\begin{aligned} \rightarrow a+1 &= \&a[0] + 1 \\ &= \&a[0] + (1 \times 4) \\ &= \underline{1004} \text{ (address of next element)} \end{aligned}$$

$$\begin{aligned} a+2 &= \&a[0] + (2 \times 4) \\ &= 1000 + 8 \\ &= \underline{1008} \end{aligned}$$

$\rightarrow (a+2)$ = memory location of `a[2]`; 1008

$* (a+i) = a[i]$

$\rightarrow *(a+2)$ = value at (memory loc: 1008) = `* &a[2];`
 $* (a+2) = a[2]$

→ Addition is commutative

$$a[i] = *(a+i) = *(1+a) = 1[a]$$

int a[4] = {10, 20, 30, 40}

10	20	30	40
1000	1004	1008	1012

printf("%d", a[2]);

printf("%d", *(a+2));

printf("%d", 2[a]);

printf("%d", *(2+a));

Output = 30

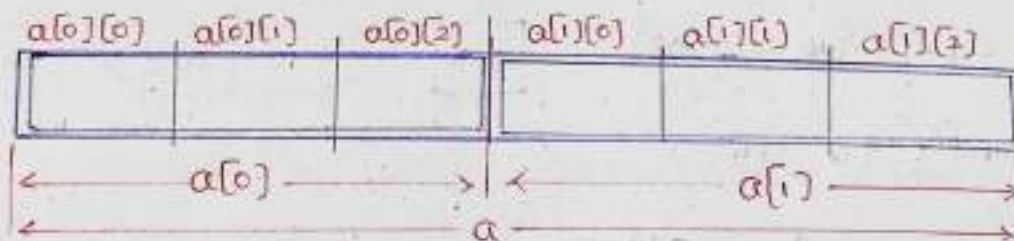
printf("%d", a+1); → 1004

printf("%d", &a+1); → &a+1 = 1000 + (1 × 16) = 1016

↓
address of
whole array

2-D Array

int a[2][3]



int a[2][3] = {1, 2, 3, 4, 5, 6}

1	2	3	4	5	6
1000	1004	1008	1012	1016	1020

→ `a` is an array of 2 elements : `a[0], a[1]`

→ `a` : array-name, address of its first element `&a[0]`

→ `a[0]` : an array of 3 elements : `a[0][0], a[0][1], a[0][2]`
it is address of its first element `&a[0][0]`

void main()
 eg: { int a[2][3] = {1, 2, 3, 4, 5, 6};
 printf("%d", a[1]); $\rightarrow \&a[0] + 1 = 1000 + (1 \times 12) = 1012$
 printf("%d", a[0][1]); $\rightarrow \&a[0][0] + 1 = 1000 + (1 \times 4) = 1004$
 printf("%d", &a + 1); $\rightarrow \&a + 1 = 1000 + (1 \times 24) = 1024$
 }
 Address of ~~three~~ whole array

eg: int a[3][4] = {1, 2, 3, 4 | 5, 6, 7, 8 | 9, 10, 11, 12} Starting address = 100

printf("%d", a);
 printf("%d", a[0]);
 printf("%d", &a); } 100
 Size of a[0] = 16 Byte
 printf("%d", a[1]); $\rightarrow 100 + (1 \times 16) = 116$
 printf("%d", a[0] + 1); $\rightarrow 100 + (1 \times 4) = 104$
 printf("%d", &a + 1); $\rightarrow \&a + 1 = 100 + (1 \times 48) = 148$

$\rightarrow a[0] + 1 = \underbrace{\&a[0][0] + 1}_{4 \text{ Byte}}$
 $= 100 + (1 \times 4)$
 $= 104$
 $= \underline{a[0][1]}$
 $\rightarrow a[0] + 2 = \&a[0][2]$

$\rightarrow *(a[0] + 1) = \text{value (memory location)} = * \&a[0][1]$
 $= \underline{a[0][1]}$

$\rightarrow *(a[1][2]) = a[1][2];$

$\boxed{*(a[i] + j) = a[i][j]}$

$\boxed{*(a[i] + j) = a[i][j]}$

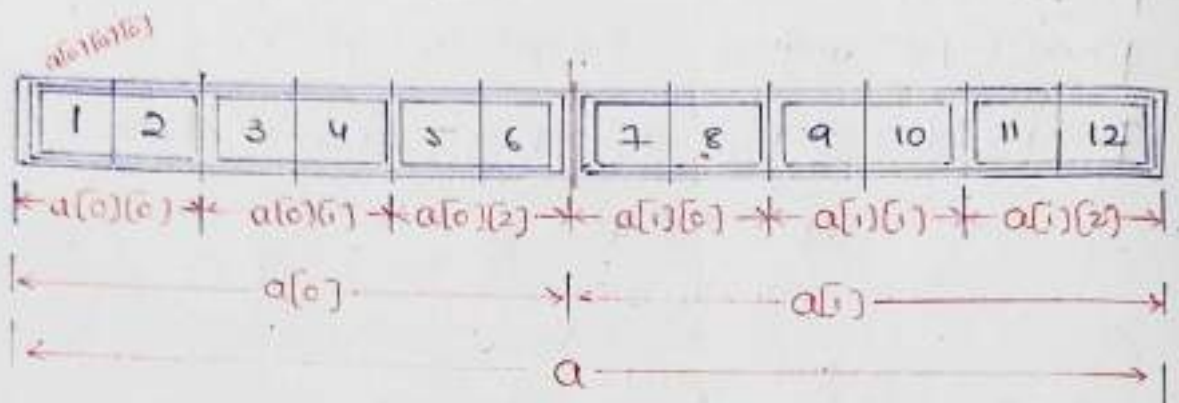
$\boxed{*(*(a+i) + j) = a[i][j]}$

a[0] $\leftarrow$ printf("%d", *a); $\rightarrow 100$
 a[0][0] $\leftarrow$ printf("%d", **a); $\rightarrow 104$
 printf("%d", *a + 1); $\rightarrow 104$

printf("%d", **a); $\rightarrow 2$
 printf("%d", *a[0]); $\rightarrow 1$

3-D Array

int a[2][3][2] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 }



`a`: array `a[0], a[1]`

`a[0]`: array `a[0][0], a[0][1], a[0][2]`

`a[0][0]`: array `a[0][0][0], a[0][0][1]`

`printf("%d", a);` → `&a[0] + 1 = 1000 + (1 * 4) = 1004`

`printf("%d", a[0]);` → 1008

`printf("%d", a[0][0]);` → 1004

`printf("%d", &a);` → 1048

Q17. int a[3][2][3] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18 }

1. `a`
 2. `a[0]`
 3. `a[0][0]`
 4. `&a`
- These four items are grouped by a bracket with the value 1000 next to it.

5. `a[0][0][0] → 1`

6. `a + 1 → 1000 + 24 = 1024`

7. `a[0] + 1 → 1000 + 12 = 1012`

8. `a[0][0] + 1 → 1004`

9. `&a + 1 → 1000 + 92 = 1092`

10. `a[0][0][0] + 1 → 2`

11. `*a → &a[0] → 1000`

12. `**a → *(&a[0][0]) → 1000`

13. `***a → *(&a[0][0][0]) = 1`

14. `*a + 1 → &a[0][0] + 1 = 1012`

15. `**a + 1 → &a[0][0][0] + 1 = 1004`

16. `*a[0] + 1 → *(&a[0][0]) + 1 = 1004`

17. `***a[0] + 1 → *(*a[0] + 1)`
 $= a[0][0][0] + 1$
 $= 1 + 1$
 $= 2$

Array declaration & initialization

→ `int a[];` × Invalid

→ `int a[10];` ✓

→ `int a[] = {10, 20, 30};` ✓ valid

→ `int a[4] = {10, 20, 30, 40};` ✓

→ If we are declaring an array without initialization, then it is mandatory to provide the size of each dimension, otherwise Compiler Error.

→ In case we are initializing an array, then we have the flexibility that we can omit first dimension size but no other dimension is having such flexibility.

`int a[] = {10, 20, 30};` ✓ valid

`int a[][2] = {1, 2, 3, 4};` ✓ valid

`int a[][2] = {1, 2, 3, 4};` × invalid

`int a[2][] = {1, 2, 3, 4, 5, 6};` × invalid.

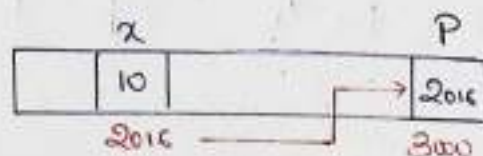
Pointers

A pointer variable is a special variable that is used to hold the address of another variable.

`int *p;` → `p` is a pointer to integer

→ `p` can store address of some integer variable.

eg. `int x = 10;`
`int *p;`
`p = &x;`



`printf("%d", x);` } 2016
`printf("%d", p);` }
`printf("%d", *p);` → 10

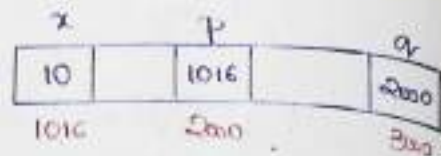
int x = 10;

int *p;

int **q;

p = &x;

q = &p;



Pointer to a pointer
of integer type

printf("%d", p); → 1016

printf("%d", *p); → 10

printf("%d", q); → 2000

printf("%d", *q); → 1016

printf("%d", **q); → 10

Assume

int - 4 byte

char - 1 byte

float - 8 byte

int *p;

char *q;

float *r;

Pointers are of
same size.

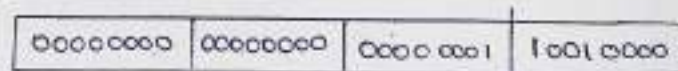
eg: int x = 300;

char *p;

p = (char*) &x;

printf("%d", *p);

O/p = -112



$$\begin{aligned} & \begin{matrix} 2^7 & 2^6 & \dots & 2^0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{matrix} \\ & = -2^7 - 2^6 - 2^5 - 2^4 - 2^3 - 2^2 - 2^1 - 2^0 \\ & = -112 \end{aligned}$$

eg: int x = 130

char *p; p = (char*) &x;

printf("%d", *p); O/p = -126 (cyclic property)

int a[] = {10, 20, 30, 40};

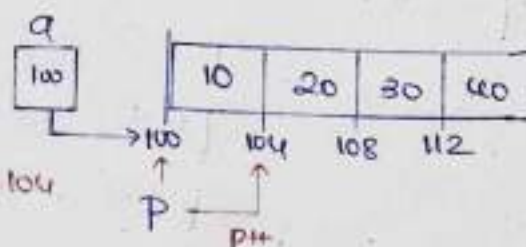
int *p;

p = &a[0];

p = p + 1; → 100 + (4) = 104

p++;
--p;

are & dec
valid ✓



Arrays and Pointers

```
int a[] = {10, 20, 30, 40};
```

```
int *p;
```

```
p = a; → p = &a[0];
```

```
printf("%d", *p10); → 10
```

```
printf("%d", p[0]); → 10
```

```
printf("%d", *(p+1)); → 20
```

```
printf("%d", p[1]); → 20
```

```
printf("%d", *(p+2)); → 30
```

$*(a+i) = a[i]$

```
int a[] = {10, 20, 30, 40};
```

$++a;$
 $--a;$
 $a++;$
 $a--;$

Invalid

$a = \{30, 40\};$ × Invalid

Array name cannot
be l-value.

```
int a[] = {10, 20, 30, 40};
```

```
int *p;
```

```
p = &a[0];
```

$++p;$
 $--p;$
 $p++;$
 $p--;$

Valid

```
p = &a[2]; ✓ valid
```

Variable

→ $\text{Pointer_var} + 3$ = moving 3 locations in forward direction.

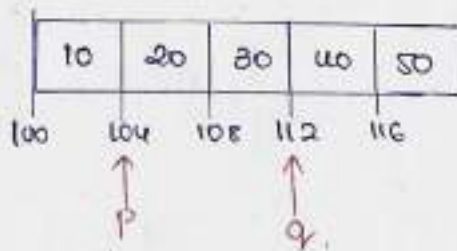
```
int a[] = {10, 20, 30, 40, 50};
```

```
int *p, *q;
```

```
p = &a[1];
```

```
q = &a[4];
```

```
printf("%d", *q - *p); ✓
```



$$\frac{\text{actual diff}}{\text{data size}} = \frac{116 - 104}{4} = \frac{12}{4} = 3$$

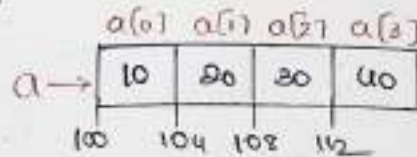
#include <stdio.h>

void main()

{ int a[4] = {10, 20, 30, 40};

int *p;

p = &a[0];



printf("%d", *p); $\rightarrow$ $*(p) = 20$

$++*p$; $\rightarrow ++(p) = 20 + 1 = 21$

$++p$; $\rightarrow$ 108 (address)

printf("%d", *p);
 $\downarrow$
30

Pass by reference / Call by reference

void main()

{ int a = 10, b = 20;

printf("a = %d, b = %d", a, b);

Swap(&a, &b); $\text{O/p: } a=20, b=10$

printf("%d %d", a, b);

}

$\text{O/p: } 20, 10$

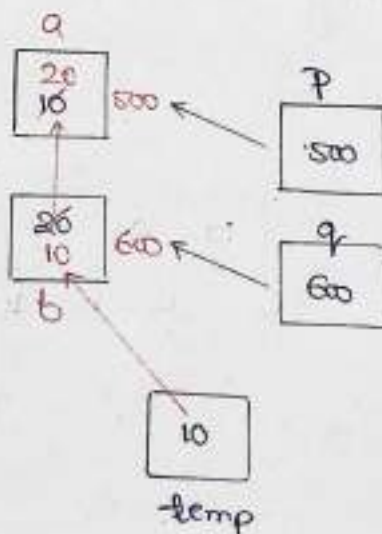
void swap(int *p, int *q)

{ int temp;

temp = *p;

*p = *q;

*q = temp; }




```

void main()
{
    int a[] = {10, 20, 30, 40};
    int sum;
    sum = fun(a, 4);
    printf("%d", sum);
}

```

O/P = 100

```

int fun(int *p, int n)
{
    int i, sum = 0;
    for (int i = 0; i < n; ++i)
    {
        sum = sum + p[i];
    }
    return sum;
}

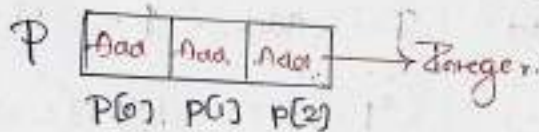
```

Complex Declarations

- | | | | | |
|---------------|---|--------------------|----------|----------|
| 1. () | → | Paranthesis | Function |] 1 Look |
| 2. [] | → | Square bracket | Array | |
| 3. Identifier | → | var name, fun name | |] 2 Read |
| 4. * | → | Pointer | | |
| 5. datatype | → | int, char, float | | 3. |

eg: ^③int ^②* ^①(p[3])

→ P is an array of 3 pointer to integer.



eg: ^③int ^①(*p) ^②[4];

→ P is a pointer to "array of 4 integer".

int a[] = {10, 20, 30, 40};

int (*p)[4];

p = &a; ✓

eg: $\text{int } (*p)(\text{int}, \text{int});$

→ P is a pointer to a function that takes 2 integer arguments and return an integer value.

eg: $\text{int } *(*p)[4];$

→ P is a pointer to an array of 4 pointer to integer

Pointer to function/function pointers

```
int Add(int a, int b)
{
    return a+b;
}
```

```
void main()
{
    int (*p)(int, int);
```

→ pointer to a function which takes 2 int arguments & return integer.

p = &Add;

printf("%d", (*p)(2, 3));

→ Calling the function with pointer.

O/p = 5.

p = Add;	P = &Add;	p = *Add;	P = &Add;
(P)(2, 3)	(*P)(2, 3)	(*P)(2, 3)	(P)(2, 3)

All are valid declarations/function calls

① $\text{int } a[4] = \{10, 20, 30, 40\};$
 $\text{int } *p[4] = \{a+2, a, a+1, a+3\};$
 $\text{int } **q;$
 $q = \&p[0];$
 $\text{printf}("%d", **q);$

O/p = 20.

$**q$
 $\downarrow$
 $p[0]$
 $\downarrow$
 $a[2] = (a[0] + 1)$
 $a[2] = 20$

② `int a[4] = {10, 20, 30, 40};`
`int *p[4] = {a[2], a, a[1], a[3]};`
`int *q;`
`q = &p[0];`
`*++*++q; --q;`
`printf("%d", *--*++q);`
 O/p = 10

③ `void fun(int*);`
`void main()`
`{ int a[4] = {10, 20, 30, 40};`
`fun(a);`
`printf("%d %d", a[0], a[1]);`
`}`
 O/p = 10 20

`void fun(int* p)`
`{ ++p;`
`*p++;`
`++(*p); }`

④ `void main()`
`{ int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};`
`fun(a);`
`printf("%d %d %d", a[1][1], a[1][2], a[2][0]); }`

`void fun(int (*p)[3])`
`{ ++p;`
`(*p)[1] = (*p)[1] + 1; }`

O/p = 6 6 7

eg: `void main()`
`{ int *p; → declaration`
`...`
`printf("%d", *p); }`
 ↓
 dereferencing
 (value fetch)

`Char *p;`
`...`
`...`
`printf("%c", *p);`
 ↓
 1 Byte info

Void Pointer

```
void *ptr; → address.
```

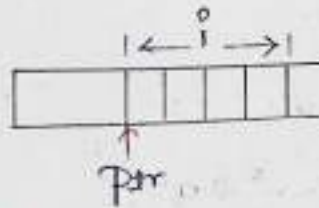
```
int i = 369;
```

```
char ch = 'A';
```

```
ptr = &i;
```

```
printf("%d", *ptr); - Error
```

```
printf("%d", *(int*)ptr);
```



```
void *p;
```

```
ptr;
```

```
p = ptr;
```

Error

* Don't try to dereference any void pointer without type casting.

* Do not apply arithmetic operations on void pointer.

```
void *ptr;
```

```
char ch = 'A';
```

```
ptr = &ch;
```

```
printf("%c", *(char*)ptr); O/p = A
```

New Pointer

* Specially designed pointer

→ `int *p = (int*)0;`

Wild Pointer

```
void main()
{
    int *q;
}
```

Garbage value

Uninitialised pointer is called a wild pointer.

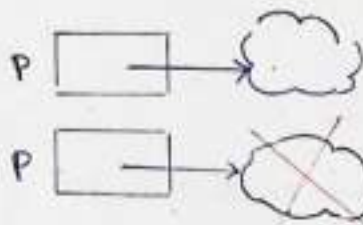
```
void main()
{
    int x = 1008;
    int *p;
    ==
    *p = 10;
}
```



if by any chance
Garbage value is 2096
then `*p = 10;` will
change the value of
`x`.
(Unintentionally)

Dangling Pointer

```
int *f()
{
    static int a=10;
    return &a;
}
```



```
void main()
{
    int *p;
```

```
    p = f();
    printf("%d", *p);
}
```

← address of a local variable, is returned but after the function is executed, the address is invalid.

error! → Solution is making 'a' static!
 Q.p = 10.

Dynamic Memory Allocation

1. malloc
 2. calloc
 3. realloc
 4. free
- Used for DMA in C.

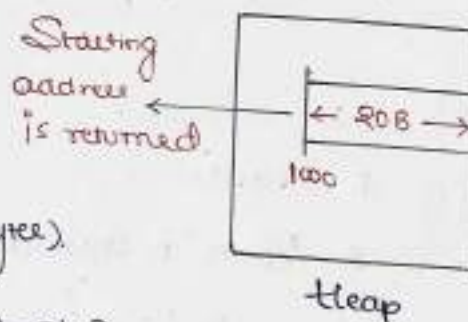
① malloc

malloc(20)

↑
Size in byte

(void*) malloc (Size in byte)

Syntax: (void*) malloc (Size & size)



sizeof

```
int *p;
p = malloc (5 * sizeof(int));
*(p+i) = p[i]
```

```
for (int i=0; i<5; ++i)
    scanf("%d", p+i);
for (int i=0; i<5; ++i)
    printf("%d", *(p+i));
```

← Storing Elements

← Printing Values

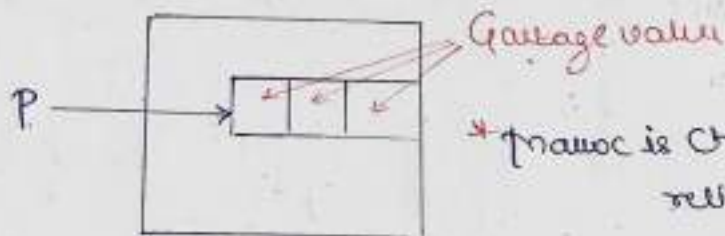
Ans: 100

void main()

```
{ int N, *p, i;  
  printf("Enter no of elements");  
  scanf("%d", &N);  
  p = malloc (N * sizeof(int));  
  if (p != NULL) {  
    for (int i=0; i<N; ++i)  
      scanf("%d", p+i);  
    for (int i=0; i<N; ++i)  
      printf("%d", p[i]);  
  }  
}
```

malloc :

1. Search
2. block is available → Starting address is returned.



* malloc is cheaper but not reliable.

② Calloc :

1. Search
2. block is available
→ Sets all bits = 0
(Instead of garbage value)
3. Starting address is returned.

* Calloc is expensive and reliable

③ realloc

Size of the dynamically allocated memory can be changed by using realloc.

```
{ int *p;
```

```
  p = malloc(5 * sizeof(int));
```

```
  p = realloc(p, 10 * sizeof(int));
```

```
}
```

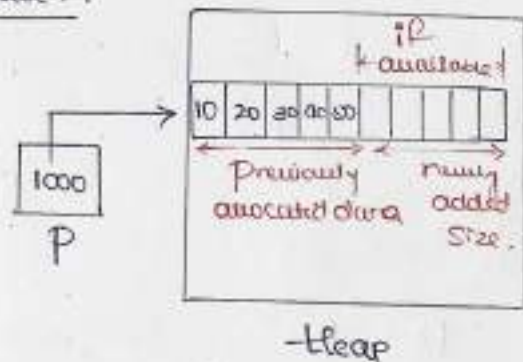
new size

```
p = malloc(5 * sizeof(int));
```

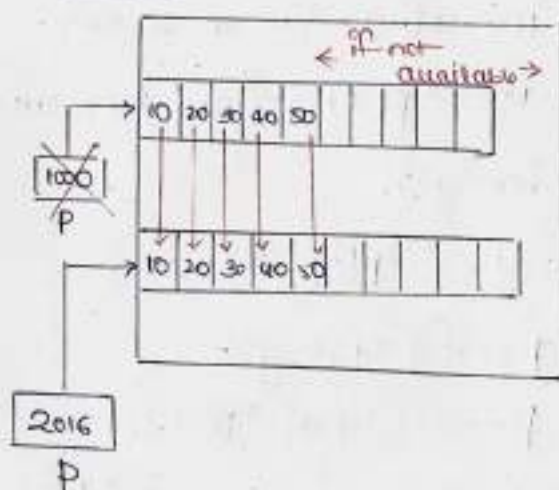
```
p = realloc(p, 3 * sizeof(int));
```

last 2 space is erased.

Case 1:



Case 2:



④ free : It is used to free the dynamically allocated memory.

```
void fc1 { int *p; }
```

```
p = malloc(100 * sizeof(int));
```

```
free(p);
```

```
void main() { fc1, fc1, fc1; }
```

* Memory allocated in heap will be there in heap throughout the lifetime of program.

* Not de-allocating the memory after use will cause "memory leakage problem", so we need to use free().

memory will not be available for other functions in heap.

Case PQs

- ① unsigned int $a[4][3] = \{\{1, 2, 3\}, \{4, 5, 6\}, \{7, 8, 9\}, \{10, 11, 12\}\};$
 $\text{printf}(" \%u \%u \%u", a[1][3], *(a[1][3]), *(a[1][2] + 3));$
 $\rightarrow 2036 \ 2036 \ 2036$

- ② $BA = 1000$, 4 byte
 $\text{int } a[2][3] = \{1, 2, 3, 4, 5, 6\};$
 $\text{printf}(" \%u \%u \%u", a, *a, **a);$
 $\text{printf}(" \%u \%u \%u", a+1, *a+1, **a+1);$
 $\rightarrow 1000 \ 1000 \ 1012 \ 1004 \ 5$

- ③ $\text{int } a[4] = \{10, 20, 30, 40\};$
 $\text{int } *p[4] = \{a+3, a+2, a+1, a\};$
 $\text{int } y;$
 $y = --p[0] - p[1];$
 $\text{printf}(" \%d", y);$
 $\text{printf}(" \%d", *p[0]);$
 $\rightarrow 0 \ 20$
 $--p[0] = \&a[3] - \&a[2]$
 $= \frac{p[0] - p[1]}{4}$
 $= \frac{\&a[2] - \&a[2]}{4} = \frac{0}{4} = 0$
 $*p[0] = *(a[2])$
 $= 30$

- ④ $\text{int } (*f)(\text{int});$

$\rightarrow$ A pointer to a function that takes an integer pointer as argument and returns an integer.

- ⑤ $\text{int } a=5, b=10, c=15;$
 $\text{int } *p[3] = \{\&a, \&b, \&c\};$
 $\text{printf}(" \%d", *p[*p[1]-8]);$
 $\rightarrow 15$
 $*p[*p[1]-8]$
 $= *p[10-8]$
 $= *p[2]$
 $= c$

⑥ void main()

{ static int a[] = {10, 20, 30, 40, 50};

static int *p[] = {a, a+3, a+5, a+1, a+2};

int **ptr = p; ptr = &p[0]

ptr++; ptr = &p[1]

printf("%d %d", ptr-p, **ptr); }

→ 1. 40

**&p[1]

*p[1] = *&a[3]
= 40

$\frac{\&p[1] - \&p[0]}{4}$

$= \frac{200 - 200}{4} = \frac{0}{4} = 0$

⑦ #define Print(x) printf("%d", x);

int x;

void Q(int z) { z = z + x; Print(z); }

void P(int *y) { int x = *y + 2;

Q(x);

*y = x - 1;
Print(x); }

void main()

{ x = 5;

P(&x);

Print(x); }

O/p → 12 22 14 7

⑧ int *A[10];

int B[10][10];

Of the following expressions which will not give CTE if used as l-value of an assignment statement.

→ A[2], A[2][2], B[2][2]

⑨ int a[4] = {10, 20, 30, 40};

int *p = (int *)(&a + 1);

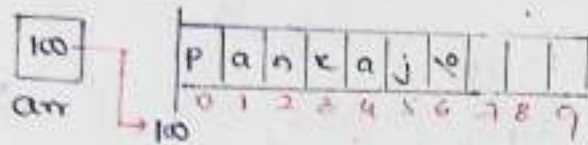
Printf("%d %d", *(a+1), *(p-1)); → 20 40

Strings

* String is a sequence of characters terminated by a null character.

Char arr[10] = "pankaj";

String constant
↓
ASCII code = 0



printf("%s", arr); → pankaj

* Size of the character array should be at least one more than length of string.

Char arr[] = {'p', 'a', 'n', 'k', 'a', 'j'};

printf("%s", arr);

Behaviour is undefined

✓ Char arr[] = {'p', 'a', 'n', 'k', 'a', 'j', '\0'};

printf("%s", arr);

① Char name[] = "pankaj";

printf("%s", name);

→ address

* String gives address to printf()

② Char name[] = {'p', 'a', 'n', 'k', 'a', 'j', '\0'};

printf("%s", name);

③ Char name[10] = "pankaj";

printf("%s", name);

```
Char name[] = "pankaj";
printf("%s", name+1);
    → ankaj
```

```
name = &name[0]
name++ = &name[0]+1
        = &name[1]
```

```
Char *ptr = "Neeraj";
printf("%s", ptr);
printf(ptr);
printf(ptr+1); → eeraj
```

```
Char arr[] = "Neeraj";
arr = "pankaj"; ×
Invalid
arr++;
--arr;
arr--;
++arr;
```

Read only area

```
Char *ptr = "Neeraj";
ptr = "pankaj"; ✓
ptr++;
ptr--;
--ptr;
++ptr;
```

* We can change content of individual element of an array.

```
arr[1] = 'A';
```

As it is stored in read/write area.

→ ptr[1] = 'a'

Do not try this
as pointers are stored
in Read only area

Array.	Pointer
* We can change individual element.	We can't change individual element.
* Cannot assign completely new string.	Can assign completely new string.
* Increment/dec not possible	Increment/decrement possible.
* Read/write area	Read only area

```

void main()
{
    printf("Hello"); // → address of 'H'
                        = Hello
    printf("Hello+1");
}

```

Address of 'e'
= Hello



eg:

```

if ("pankaj"[2] == "neeraj"[0])
    printf("yes");
else printf("No");

```

eg: Char arr[] = "GATE";

```

printf("%s", arr);
printf("arr");
printf(arr+1); // → ATE
printf(arr+3); // → TE

```

pf(arr + 3[arr] - arr[1])

arr + 3 - 1 = 'A'

arr + 4 - 1 = 'E'

arr + 4 = '\0'

∴ no output

eg: Char arr[] = "GATE 2023";

Char *ptr;

ptr = arr+3;

ptr = '\0'; → arr+3 = 0

printf(arr); → GATE

printf(ptr); → no output

= GATE\0...

String ends here.

① Char arr1[] = "pankaj";
Char arr2[] = "pankaj";

```

if (arr1 == arr2) printf("yes");
else printf("No");

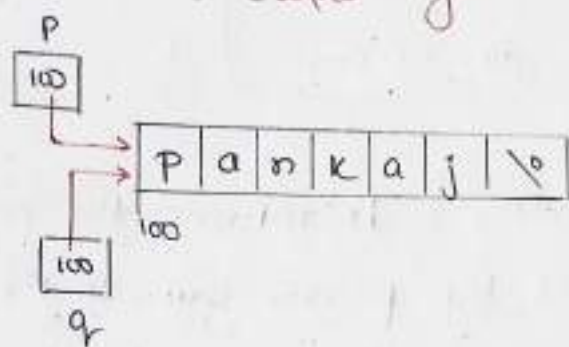
```

both have different address
Array name is address of
first element
So Output = No

② Char arr1[] = "pankaj";
 Char arr2[] = "pankaj";
 if (*arr1 == *arr2) pf("yes");
 else pf("No");

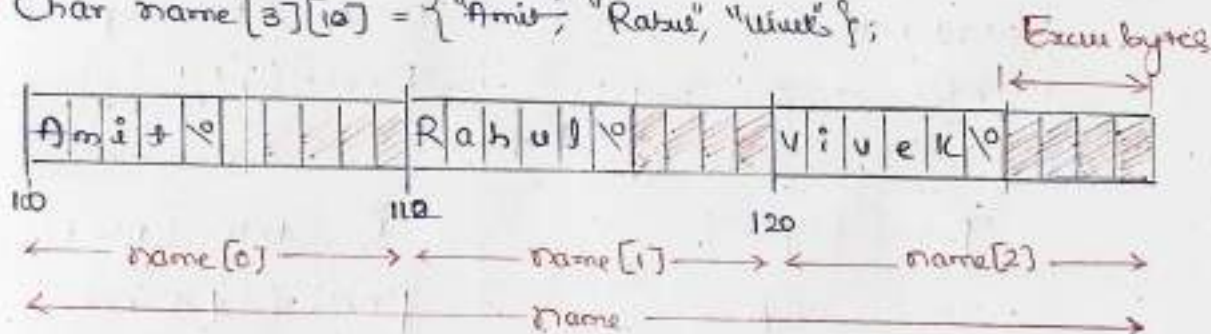
*arr1 = *arr1[0]
 = 'p'
 *arr2 = *arr2[0]
 = 'p'
 ∴ output = "yes".

③ Char *p = "pankaj";
 Char *q = "pankaj";
 if (p == q) pf("yes");
 else printf("No");



Multiple Strings

→ Char name[3][10] = {"Amit", "Rahul", "Vivek"};



printf("%s", name[0]); → Amit

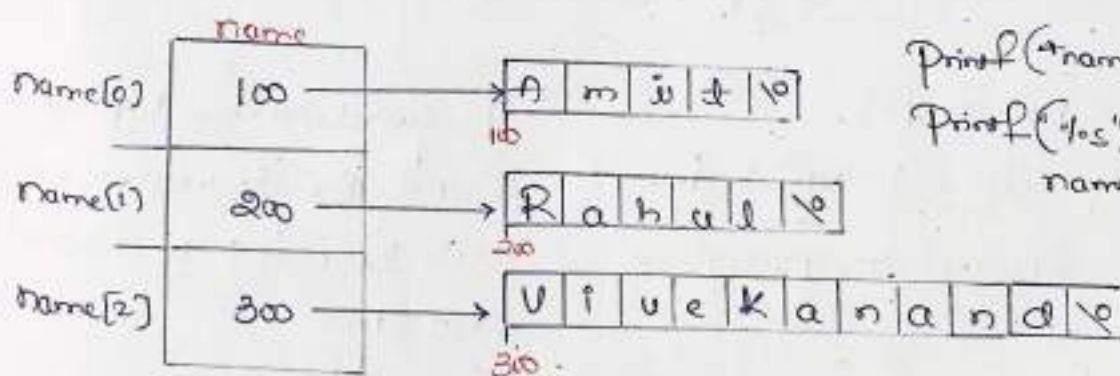
printf("%s", name[0]+2); → it

*(name[1]+2)

↓
 name[1][2] = 'h'

* Drawback using 2D array is, some bytes will remain unused.

→ Char *name[3] = {"Amit", "Rahul", "Vivekanand"};



printf(*name)

printf("%s",
 name);

} Amit

<string.h>

(i) Strlen

(ii) Strcpy

(iii) Strcat

(iv) Strcmp

String Functions on String

① Strlen : It returns the no. of symbols in the string pointed by pointer given to this function ('\\0' is not counted).

→ unsigned int Strlen (char *p);

eg: #include <string.h>

void main()

{ char arr[10] = "pankaj";
int i = Strlen(arr);
printf("%d", i); }

Op = 6

Char *ptr = "pankaj";

int i = Strlen(ptr);

printf("%d", i);

→ 6

i = Strlen(ptr + 2);

printf("%d", i);

→ 4

② Strcpy : It copies the string pointed by the source pointer to the memory/buffer pointed by destination pointer. (also copy '\\0').

→ Char *p strcpy (Char *destination, const Char *source);

eg: Char arr[10];

arr = "pankaj"; x Invalid

strcpy(arr, "pankaj"); ✓

* To avoid overflow type error
Size of the destination memory
must be enough to accommodate
new string.

eg: Char arr[10];
 Char *ptr = "pankaj";
 strcpy(arr, ptr);
 printf("%s", arr); $\rightarrow$ Pankaj.

③ Strcat: It appends one string at the end of other string

$\rightarrow$ Char* Strcat(Char* destination, const Char* source);

It must be
an array

It appends string pointed
by source pointer at the end
of string pointed by
destination pointer.

eg: Char arr[20] = "Pankaj";
 Char *ptr = "Sharma";
 Strcat(arr, ptr);
 printf("%s", arr); $\rightarrow$ Pankaj Sharma.

④ Strcmp: It compares two strings character by character
 and returns 0 if both are same.

$\rightarrow$ Strcmp(String1, String2);

Char* ptr1 = "pankaj";

Char* ptr2 = "pani";

$\begin{cases} 0 \rightarrow \text{Same} \\ < 0 \rightarrow \text{Smaller} \\ > 0 \rightarrow \text{greater} \end{cases}$

Strcmp(ptr1, ptr2)

$\downarrow \quad \downarrow$
 'k' 'i'

$k - i = 102 > 0$

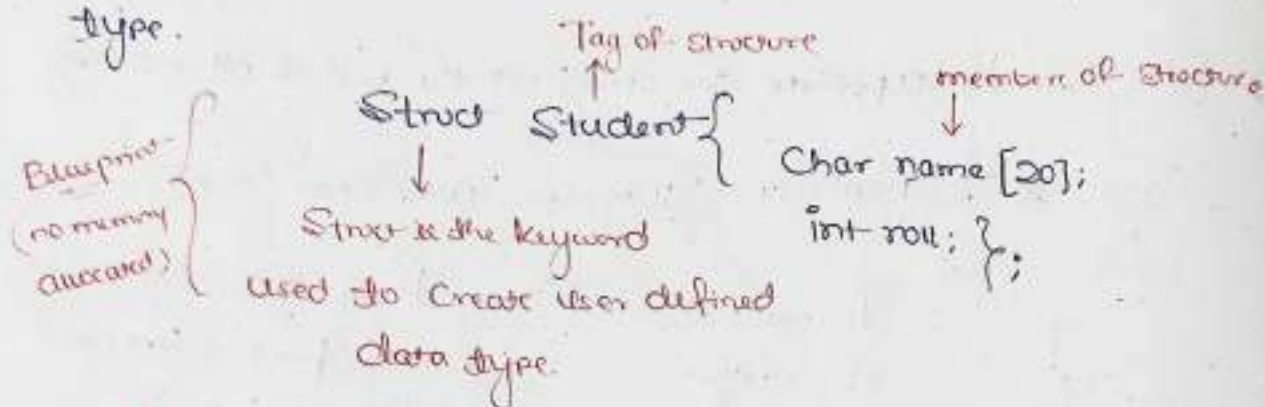
Strcmp(ptr2, ptr1)

$\downarrow \quad \downarrow$
 'i' 'k'

$i - k = -102 < 0$

Structures and Union

Structure: Collection of heterogeneous type / different type of data elements is called structure. It is a user defined data type.



* When we create a structure, there can be new datatype like `int`, `float`, `char`.

→ Syntax to use: struct student name;

eg:

```
struct Student {  
    char name[20];  
    int roll; }
```

 } **Structure blueprint / template (Global)**

```
void main()  
{  
    struct Student S;  
    S.roll = 10; }
```

• ⇒ membership Operator

Ways of making Structure

①

```
struct {  
    char name[20];  
    int roll; } S1, S2;
```

Annotations:

- no tag**: points to the `struct` keyword
- Variable to use**: points to `S1, S2`
- we can't create a variable**: points to the curly braces

```
void main()  
{  
    // we can't create a variable  
}
```

②

```
typedef struct Student  
{  
    char name[20];  
    int roll; } X;
```

```
void main()  
{  
    X S1;  
    or  
    struct Student S1;  
}
```

③ Struct Student { char name[20];
int roll; }

void f() { $\rightarrow$ default value = 0 or null

Struct Student s1;

};

void main() {

Struct Student s2;
};

$s2 = \{10, "pankaj"\};$ $\times$ Invalid/Error
wrong order

$s2 = \{"pankaj", 10\};$ $\checkmark$ Order matters (of datatype)

$s3 = \{"pankaj"\};$ $\checkmark$

$s2.name = "pankaj";$ $\times$ Invalid

$\rightarrow$ name is array, so 1-value can't be constant.

Solution $\downarrow$

Struct Student s;

strcpy(s.name, "pankaj"); $\checkmark$

s.roll = 10; $\checkmark$

eg: Struct Student { char name[20];
int roll; }

void main() {

Struct Student s[2];

s[0].roll = 10; s[1].roll = 20; $\checkmark$

s[0].name = "pankaj"; $\times$ Invalid/Error

strcpy(s[0].name, "pankaj"); $\checkmark$

strcpy(s[1].name, "Neeraj"); }

Struct Student s1 = s[0]; $\checkmark$ valid

Struct Student
 $s1 = \{10, 10\};$

Struct Student
 $s2 = s1;$ $\checkmark$
valid

```

Struct dateofbirth { int day;
                    int month;
                    int year; } } - template

```

```

Struct Student { Char name[20]; -> derived
                int roll;       -> primitive

```

```

Struct dateofbirth dob; } -> user defined data type

```

Structure inside Structure.

```

void main()
{
    Struct Student s;
    strcpy(s.name, "Pankaj");
    s.roll = 20;
    s.dob.day = 2;
    s.dob.month = 5;
    s.dob.year = 1982; }

```

accessing member of dob

eg: Struct Student {
 Char name[20];
 int roll; }

```

void main()
{
    Struct Student s;
    strcpy(s.name, "Pankaj");
    s.roll = 10;
    display(s); }

```

(call by value)

```

display(&s);

```

(call by reference)

```

void display (Struct Student s)
{
    printf("%s", s.name);
    printf("%s", s.roll); }

```

```

void display (Struct Student
              pointer to Structure -> *ptr)
{
    printf("%s", (*ptr).name);
}

```

"Pankaj"

→ $(*ptr).member1$ → ptr is a pointer to structure & we can access $member1$ like this.

→ ptr : pointer to structure

Syntax: $(*ptr).member1$ or $ptr \rightarrow member$.

Self Referential Structure.

```
struct Pankaj { int i;
```

```
    struct Pankaj *ptr; };
```

```
void main()
```

```
{ struct Pankaj s1, s2, s3;
```

```
    s1.i = 10;
```

```
    s2.i = 20;
```

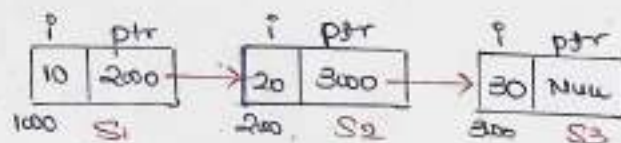
```
    s3.i = 30;
```

```
    s1.ptr = &s2;
```

```
    s2.ptr = &s3;
```

```
    s3.ptr = NULL; }
```

Member which is pointer to same data type.



Union: Union is a keyword used to create user defined data type. Similar to structure.

* In case of Structure, all members get individual memory space but all members of a union variable share a common memory area.

Syntax: union my { int a;
char c; }

Struct A
 { Char i; $\rightarrow$ 1 Byte
 int j; $\rightarrow$ 4 Byte
 };

void main()
 { Struct A a;
 printf("%d", sizeof(a));
 O/P = 5

Union B
 { Char i; $\rightarrow$ 1
 int j; $\rightarrow$ 4
 };

void main()
 { printf("%d", sizeof(b));
 O/P = 4. (union B b;)

$\max(1, 4) = 4$

- * In Union at a time you can work with only one of its members because memory is shared among all the members.
- * Memory/Size of Union is equal to its largest member size.
- * Application of Union is "type punning";

Miscellaneous

Comma Operator

* It works as a Separator

* It has the least priority as operator.

* It works as an Operator.

→ `int x=10, y=20, z=30;`

or

`int x=10;`

`int y=20;`

`int z=30;`

`a = (10, 20, 30);` `var = (Exp1, Exp2, ...)`

All these expressions are evaluated from left to right and the final value of variable will be the rightmost expression value. Expressions before the last one are evaluated and simply discarded.

eg. `int i;`

`i = (printf("pankaj"), 10+3);` last printf

`printf("%d", i);` → `pankaj 13.`

13.

during evaluation

Scoping

Static / lexical
Scoping

Decision → Compile
time

Dynamic Scoping

Scope related decision → Runtime

→ `int a;`

`{ { {`
`{ } }`

`printf("%d", a);`

→ Consider the program in a hypothetical language that allows global variable and a choice of static & dynamic scoping.

int i;

Program main()

```
{ i = 10;  
  call f(); }
```

```
Procedure f() { int i = 20;  
               call g(); }
```

```
Procedure g() { print(i); }
```

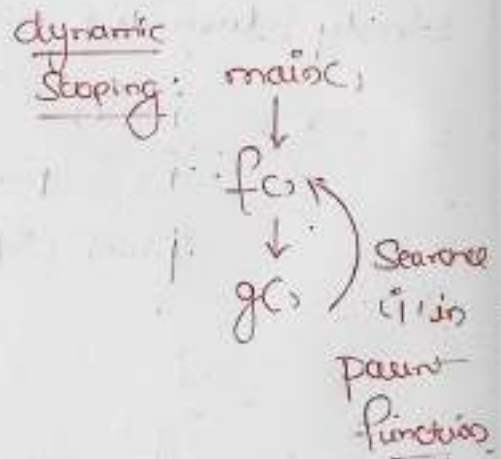
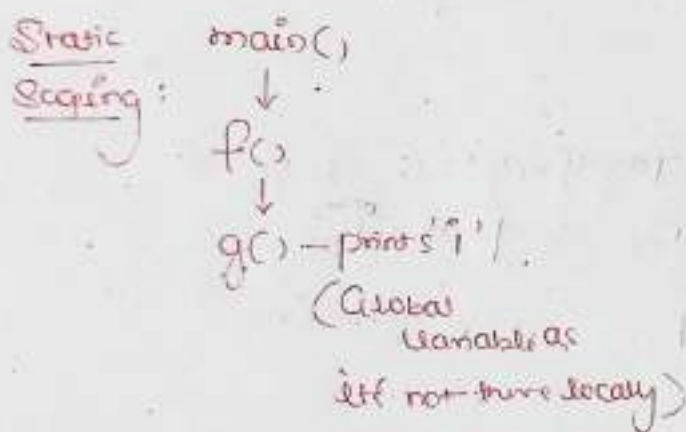
With static: O/P = 10 (Global Variable)

With dynamic

Scoping: O/P = 20.

(Searches i's value from parent function)

Let a: value under static & dynamic scoping



Preprocessor Directive

#include <stdio.h> → file inclusion

#define MAX 10 → macro, every occurrence of MAX is replaced by 10

```
void main()
{ printf("%d", MAX); }
```

→ In preprocessor this line is

⇒ printf("%d", 10);

#define Square(x) x*x;

void main()

```
{
    int i;
    i = Square(513);
    printf("%d", i);
}
```

$i = 513 * 513$
 $= 263$

O/p = 263.

Size of Operator

* Compile-time Operator

* Unary Operator $\Rightarrow$ 1 Operand

Variable

data type

Expression

Constant/literal

$i = 4$
 $char = 1$

int i = 2, j = 3;

pf("%u", sizeof(i)); $\rightarrow 4$

pf("%u", sizeof(i + 2 * j)); $\rightarrow i + i + i + i = i + i + i + i = 4$

pf("%u", sizeof(2));

pf("%u", sizeof(int));

} 4..

eg: int i = 1, j;
 j = sizeof(++i); $\rightarrow$ not executed, as sizeof is compile-time operator & $i++ = i$

printf("%d %d", i, j);

$\Rightarrow 1 4$

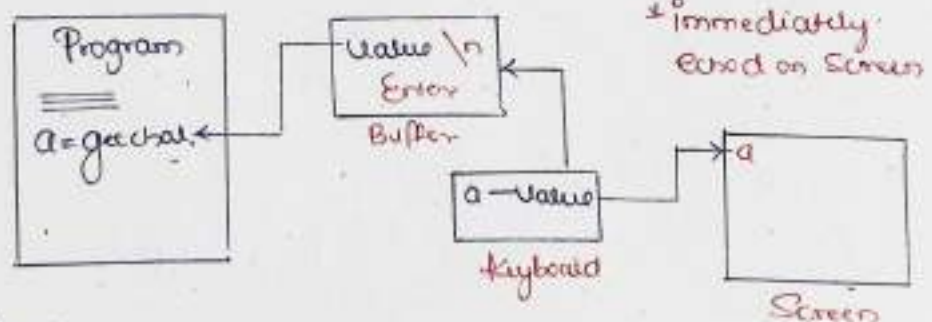
printf("%d", sizeof i); valid

no parenthesis $\checkmark$ O/p = 4.

getchar :

① buffer

② Echo.



getch :

① unbuffered

② unechoed. $\rightarrow$ nothing echo on screen.